MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

SELETIVA AULA 9

Hashing

Árvore de Prefixos

LCP

Pi Array

Caio Santos <cns>
Toni Amorim <tca>
Ryei Estevan <resm>

rev. 1.0

CC BY NC SA

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

Hashing

MaratonaCIn

Problema

MaratonaCIn Centro de Informática UFPE Universidade Federal de Pernambuco

String Matching

TASK | STATISTICS

Time limit: 1.00 s **Memory limit:** 512 MB

Given a string and a pattern, your task is to count the number of positions where the pattern occurs in the string.

Input

The first input line has a string of length n, and the second input line has a pattern of length m. Both of them consist of characters a–z.

Output

Print one integer: the number of occurrences.

Constraints

- $1 \leq n, m \leq 10^6$

Problema

MaratonaCIn · Centro de Informática UFPE · Universidade Federal de Pernambuco

```
string s, p; cin >> s >> p;
int ans;
int fim = s.size() - p.size();
for(int i = 0; i < fim; i++){
  if(s.substring(i, p.size()) == p){
    ans++;
  }
}
```

- Qual a complexidade deste código?
- (N - M) comparações
- Cada comparação linear em M
- O(NM) ≃ 10^12

Ideia

- Se comparar strings é custoso, é possível transformá-la em algo mais rápido de comparar?

$$\text{Hash}(S) = \left(\sum_{i=1}^{n} s_i \right)$$

- **Ideia 1:** representar a string como a soma de valores associados a seus caracteres.
  - Não preserva ordem
  - Valores previsíveis: fácil de quebrar

Ideia

- **Ideia 2:** adicionar um termo K que depende da posição do caractere.

$$\text{Hash}(S) = \left(\sum_{i=1}^{n} s_i * k^i \right) |k >= |alfabeto|$$

  - Hash(S) == Hash(P) ⇔ S == P (basicamente um sistema numérico)
  - K cresce exponencialmente: como guardar esse valor?

Ideia

- **Ideia 3:** limitar em valores viáveis.

$$\text{Hash}(S) = \left(\sum_{i=0}^{n-1} s_i \cdot k^{n-1-i} \right) \mod M$$

  - Colisões
  - Atualização da potência de K
  - M e K primos
    - Evitar aparição de 0
    - Distribuição mais uniforme possível

Colisões

MaratonaCIn Centro de Informática UFPE Universidade Federal de Pernambuco

- Como lidar com colisões?

$$P(s \neq t \wedge \text{Hash}(s) = \text{Hash}(t)) \approx \frac{1}{M}$$

- N strings geram (N*(N-1))/2 comparações possíveis
- Mesmo com M na casa de 1e9 precisa de menos de 100.000 strings para tornar uma colisão extremamente provável.
- Solução: **hash duplo**

Código

- prefix_hashX[N] guarda o hash da string inteira com PX E MX
  - Utilizar o par {hash1, hash2} para comparações
- No entanto, precisamos não só do hash da string P inteira como também calcular o hash de todas as substrings de S de forma eficiente
- Complexidade: O(N) construção, O(1) comparação

```
static const ll P1 = 31;
static const ll P2 = 37;
static const ll M1 = 1e9 + 33;
static const ll M2 = 1e9 + 93;
Hasher(const vector<ll>& s) {
    n = s.size();


    for (int i = 0; i < n; i++) {
        ll val = s[i];
        prefix_hash1[i + 1] = (prefix_hash1[i] * P1 + val) % M1;
        prefix_hash2[i + 1] = (prefix_hash2[i] * P2 + val) % M2;
    }
}
```

Substrings

$$\text{Pref}(0) = s_0(modM)$$

$$\text{Pref}(r) = \text{Pref}(r-1) \cdot k + s_r(modM)$$

$$\text{Hash}(l..r) = Pref(r) - (Pref(l-1) * k^{(r-l+1)})(modM)$$

```
Hash substr_hash(int i, int j) {
    int len = j - i + 1;

    ll h1 =
        (((prefix_hash1[j + 1] - (prefix_hash1[i] * powers1[len]) % M1) %
          M1) +
         M1) %
        M1;

    ll h2 =
        (((prefix_hash2[j + 1] - (prefix_hash2[i] * powers2[len]) % M2) %
          M2) +
         M2) %
        M2;

    return {h1, h2};
}
```

Problema

MaratonaCIn Centro de Informática UFPE Universidade Federal de Pernambuco

Problem Statement

You are given integer sequences $A = (a_1, \ldots, a_N)$ and $B = (b_1, \ldots, b_N)$, each of length N.

For $i = 1, \ldots, Q$, answer the query in the following format.

- If the set of values contained in the first x_i terms of A, $(a_1, \ldots, a_{x_i})$, and the set of values contained in the first y_i terms of B, $(b_1, \ldots, b_{y_i})$, are equal, then print `Yes`; otherwise, print `No`.

Constraints

- $1 \leq N, Q \leq 2 \times 10^5$
- $1 \leq a_i, b_i \leq 10^9$
- $1 \leq x_i, y_i \leq N$
- All values in input are integers.

Hash de “sets”

- **Parte 1:** A ordem não importa
- Podemos só remover o termo k^i que indicava a ordem no hash original?
- Colisões se tornam fáceis demais: {1, 4} == {2, 3}, muitos termos linearmente dependentes juntos.
- Resolvendo: **“espalhar” números no espaço**
- Utilização do XOR

```
mt19937 rng((int) chrono::steady_clock::now().time_since_epoch().count());

//pode gerar ints ou doubles, basta mudar o parametro do template
long long uniform(long long l, long long r){
    uniform_int_distribution<long long> uid(l, r);
    return uid(rng);
}
map<long long, long long> valor;
for(auto num : input){
  if(!valor.count(num)){
    valor[num] = uniform(1, 1e18);
  }
}

long long hsh = 0;
for(int i = 0; i < n; i++){
  hsh ^= valor[input[i]];
}
```

Hash de “sets”

- **Parte 2:** Só importam valores únicos
- $((X \oplus A) \oplus X = A$
- A contribuição do X no hash depende de somente de sua paridade no intervalo
  - $\{1, 2\} \neq \{1, 1, 2\}$
- Resolvendo: como **são sempre prefixos**, basta **considerar a primeira aparição**
- Complexidade: O(N)

```cpp
map<long long, long long> valorB = valorA
for(int i = 1; i <= n; i++){
  prefxorA[i] = prefxorA[i-1] ^ valorA[a[i-1]];
  valorA[a[i-1]] = 0;
  prefxorB[i] = prefxorB[i-1] ^ valorB[b[i-1]];
  valorB[b[i-1]] = 0;
}

while(q--){
    cin >> x >> y;
    if(prefxorA[x] == prefxorB[y]){
        cout << "Yes" << endl;
    } else{
        cout << "No" << endl;
    }
}
```

Hash de “sets”

- E se o (l, r) não for somente prefixos?
- Resolvendo: **fazer offline**
- Usar uma Segtree ou BIT para remover o elemento repetido e pegar soma em range rapidamente
- Complexidade: O(Nlog(N))

```
for(int i = 1; i <= n; i++){
  hash = valor[input[i]];
  if(visto.count(hash)){
    hashed[visto[hash]] ^= hash;
  }
  visto[hash] = i;
  hashed[i] = hash;
  //queries com r==i
  for(auto q : queries[i]){
    ans[q.index] = xorsum(q.l, i);
    //somatorio xor do vetor hashed
    //pode usar uma bit ou seg
    //fazendo em logn
  }
}
```

Problema

MaratonaCIn Centro de Informática UFPE Universidade Federal de Pernambuco

Problem Statement

You are given sequences of positive integers of length N: $A = (A_1, A_2, \ldots, A_N)$ and $B = (B_1, B_2, \ldots, B_N)$.

You are given Q queries to process in order. The i-th query is explained below.

- You are given positive integers l_i, r_i, L_i, R_i. Print `Yes` if it is possible to rearrange the subsequence $(A_{l_i}, A_{l_i+1}, \ldots, A_{r_i})$ to match the subsequence $(B_{L_i}, B_{L_i+1}, \ldots, B_{R_i})$, and `No` otherwise.

Constraints

- $1 \leq N, Q \leq 2 \times 10^5$
- $1 \leq A_i, B_i \leq N$
- $1 \leq l_i \leq r_i \leq N$
- $1 \leq L_i \leq R_i \leq N$
- All input values are integers.

Hash de “multisets”

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

- Ordem não importa
- Frequência importa
- XOR não é mais viável
- Basta somar
- Colisões um pouco mais prováveis: lembrar de hash duplo

```
for(int i = 1; i <= n; i++){
  prefixhash[i] = prefixhash[i-1] + valor[input[i]];
}

for(int i = 0; i < q; i++){
  cin >> l >> r;
  cout << prefixhash[r] - prefixhash[l-1] << endl;
}
```

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

Árvore de Prefixos

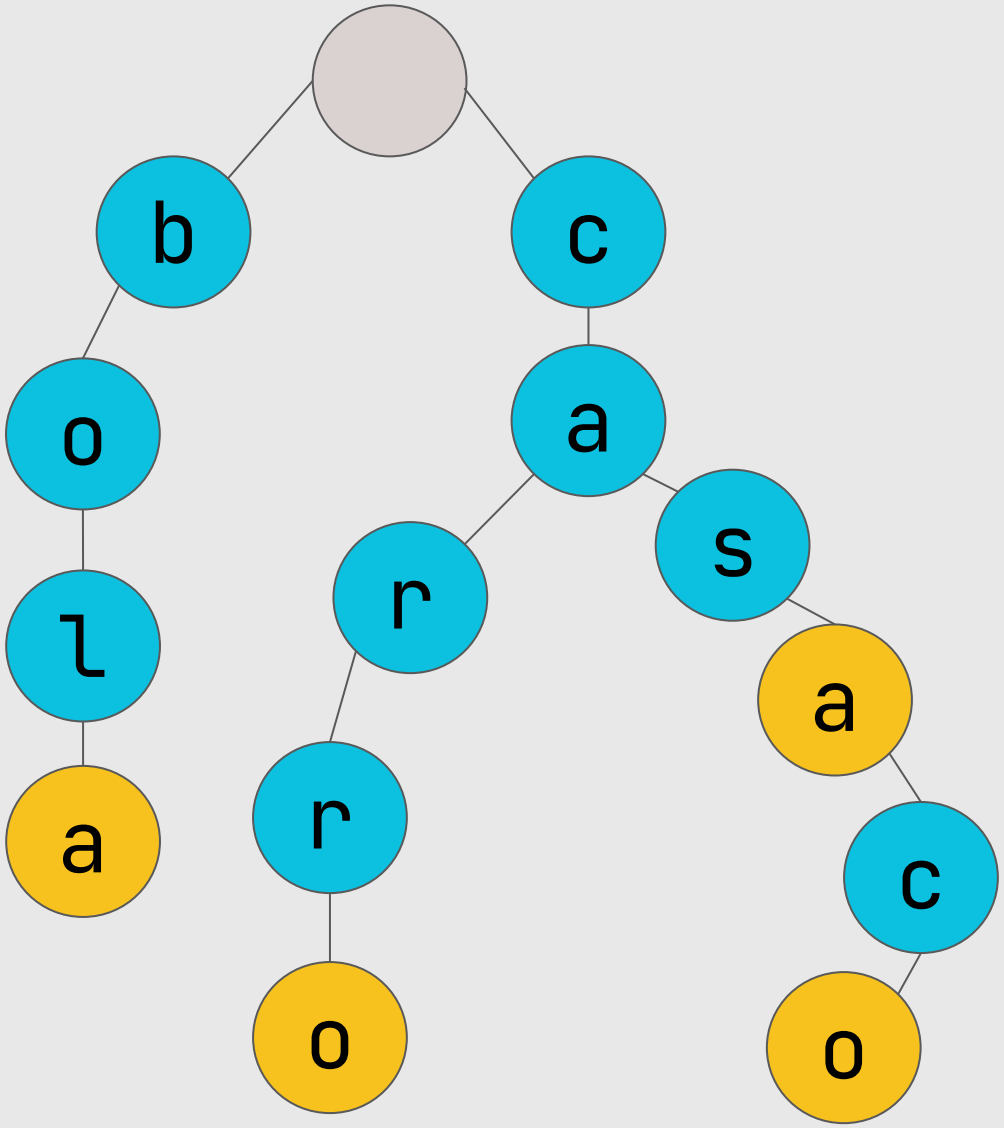
MaratonaCIn

TRIE

Uma TRIE, também conhecida como árvore de prefixos, é uma estrutura de dados semelhante a uma árvore usada para armazenar um conjunto de strings de forma eficiente, permitindo buscas rápidas por prefixos.

Ela organiza os dados hierarquicamente, facilitando a pesquisa de palavras com prefixos comuns. Cada nó na trie representa um caractere, e os caminhos da raiz até os nós formam os prefixos das palavras armazenadas.

A trie ao lado contêm as palavras “bola”, “carro”, “casa” e “casaco”. Os nós em dourado representam o final de uma palavra. Note que ela está ordenada em ordem lexicográfica.

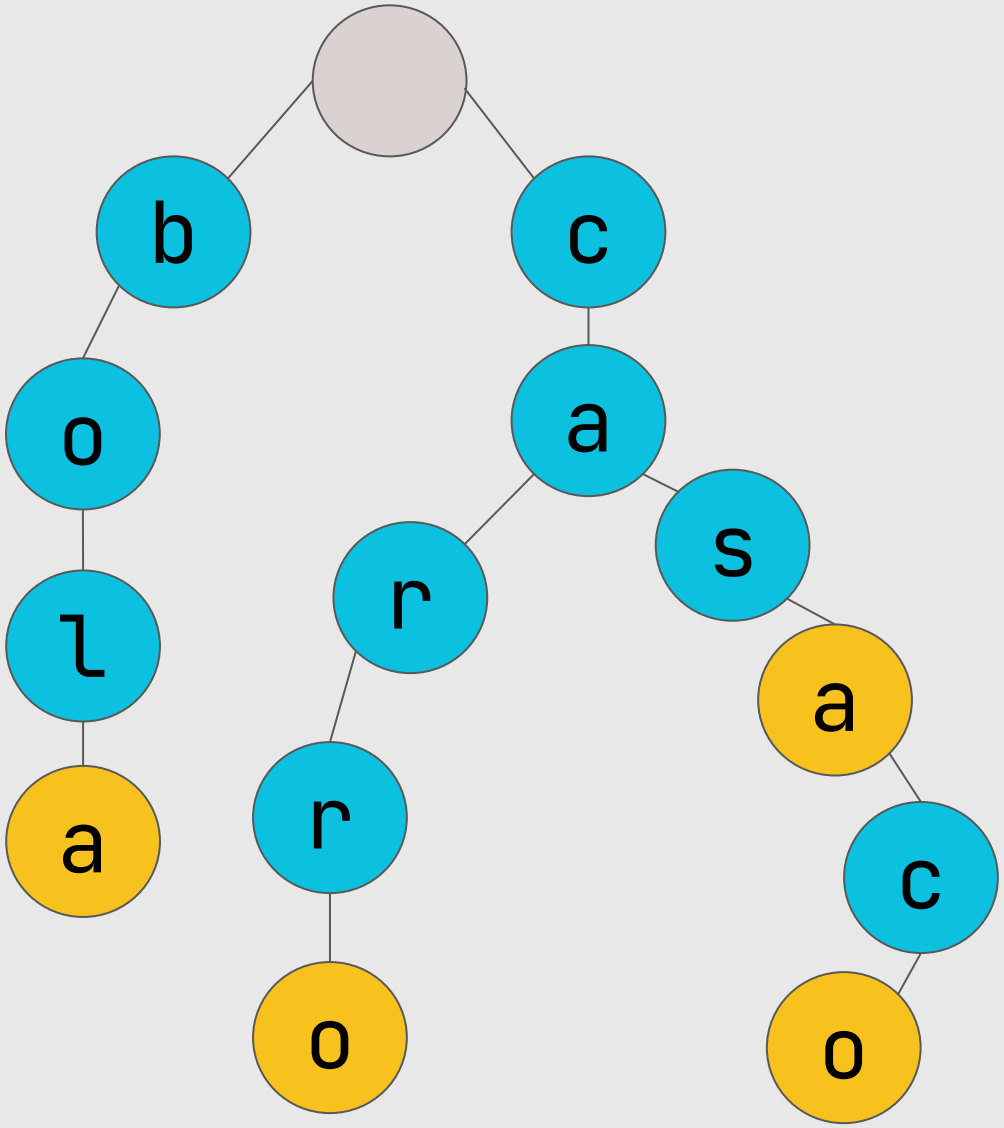


TRIE

Cada nó da TRIE precisa armazenar o seu próprio valor, um booleano indicando se ele é um nó terminal e um array ponteiros para os próximos nós.

Se soubermos previamente o tamanho do alfabeto Σ, podemos implementar o array de ponteiros com eficiência temporal O(1), utilizando um hashing perfeito entre as chaves e os inteiros de 1 até |Σ|.

No entanto, caso o tamanho do dicionário seja desconhecido, ou para economizar memória, podemos implementar cada “array” de ponteiros como uma árvore, executando operações em O(logn).

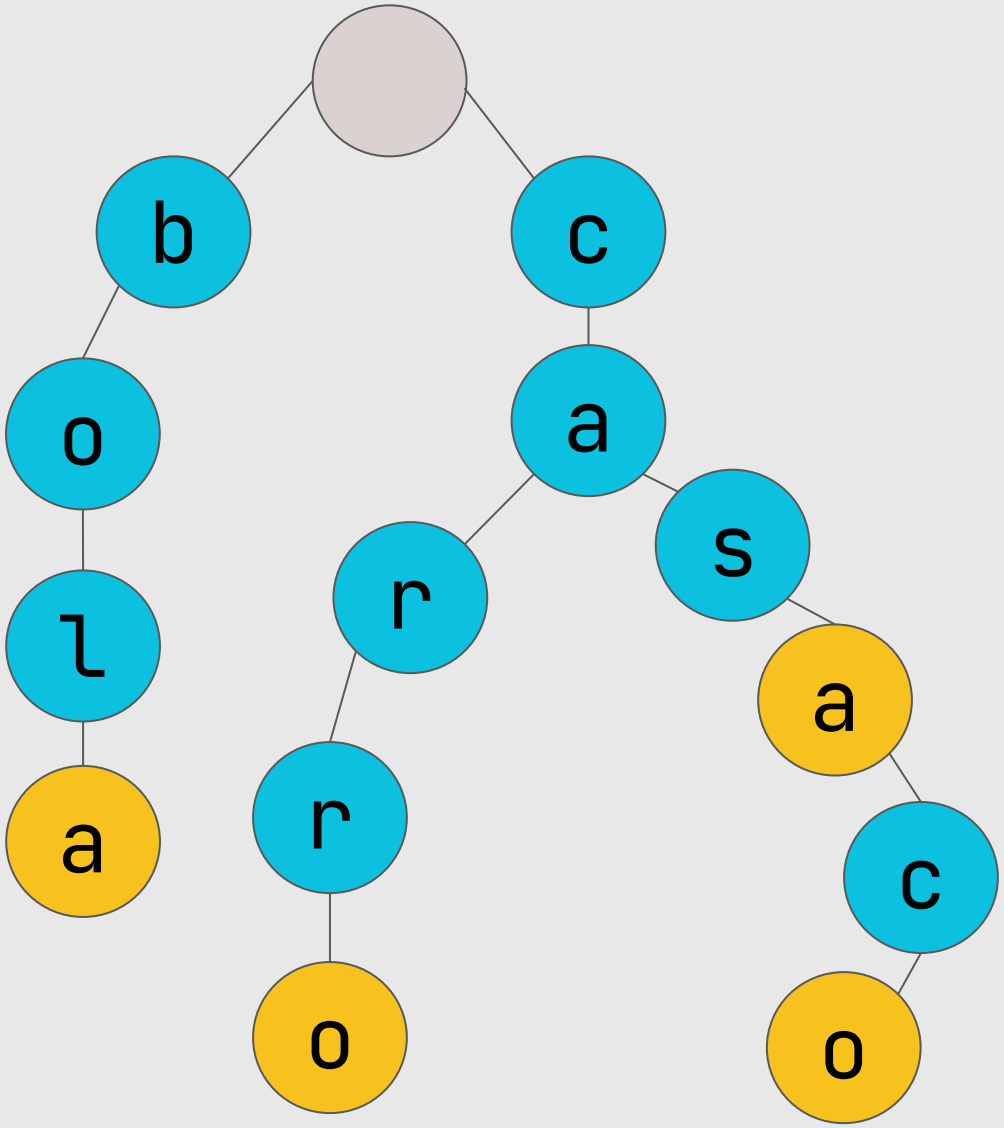


TRIE

Para fazer buscas em uma TRIE, precisamos apenas partir da raiz e navegar pelos nós desejados até que se descubra se é a palavra desejada está dentro da trie ou não.

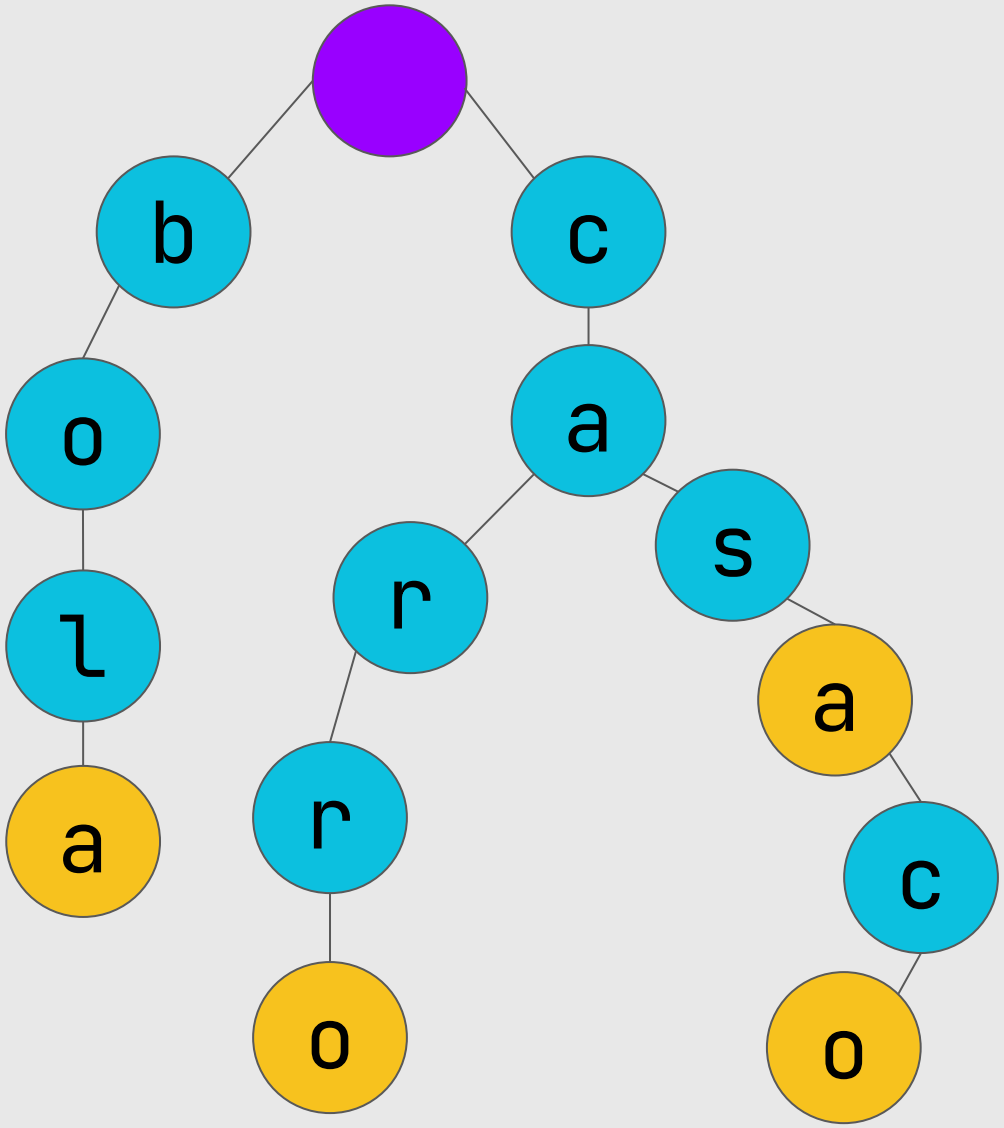
Isso é feito em no máximo o número de caracteres da string que está sendo pesquisadas.

Caso os nós da TRIE estejam O(1), a busca será em O(n). Caso os nós da TRIE estejam O(logn), então a busca será em O(n*logn).



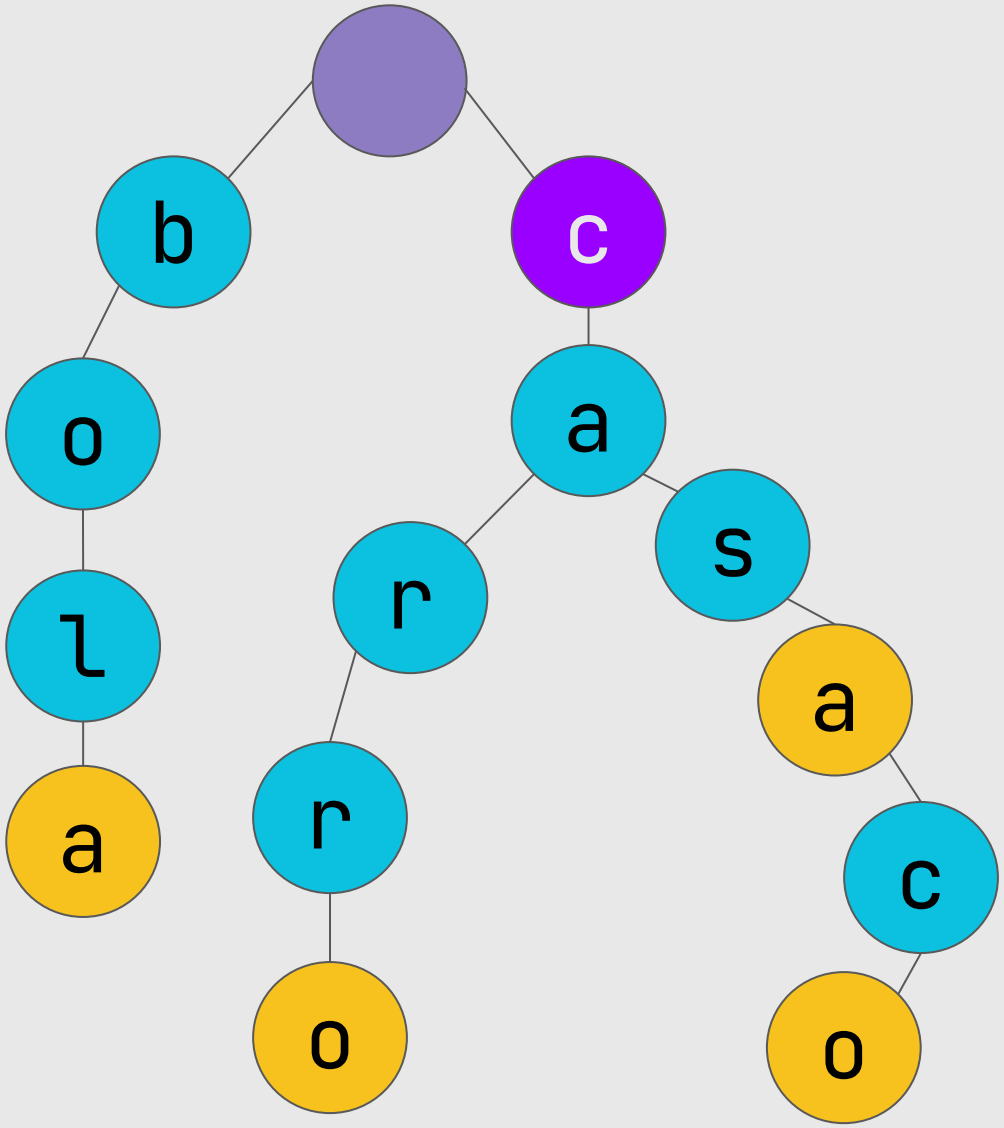
TRIE

Busca da string “carro”:



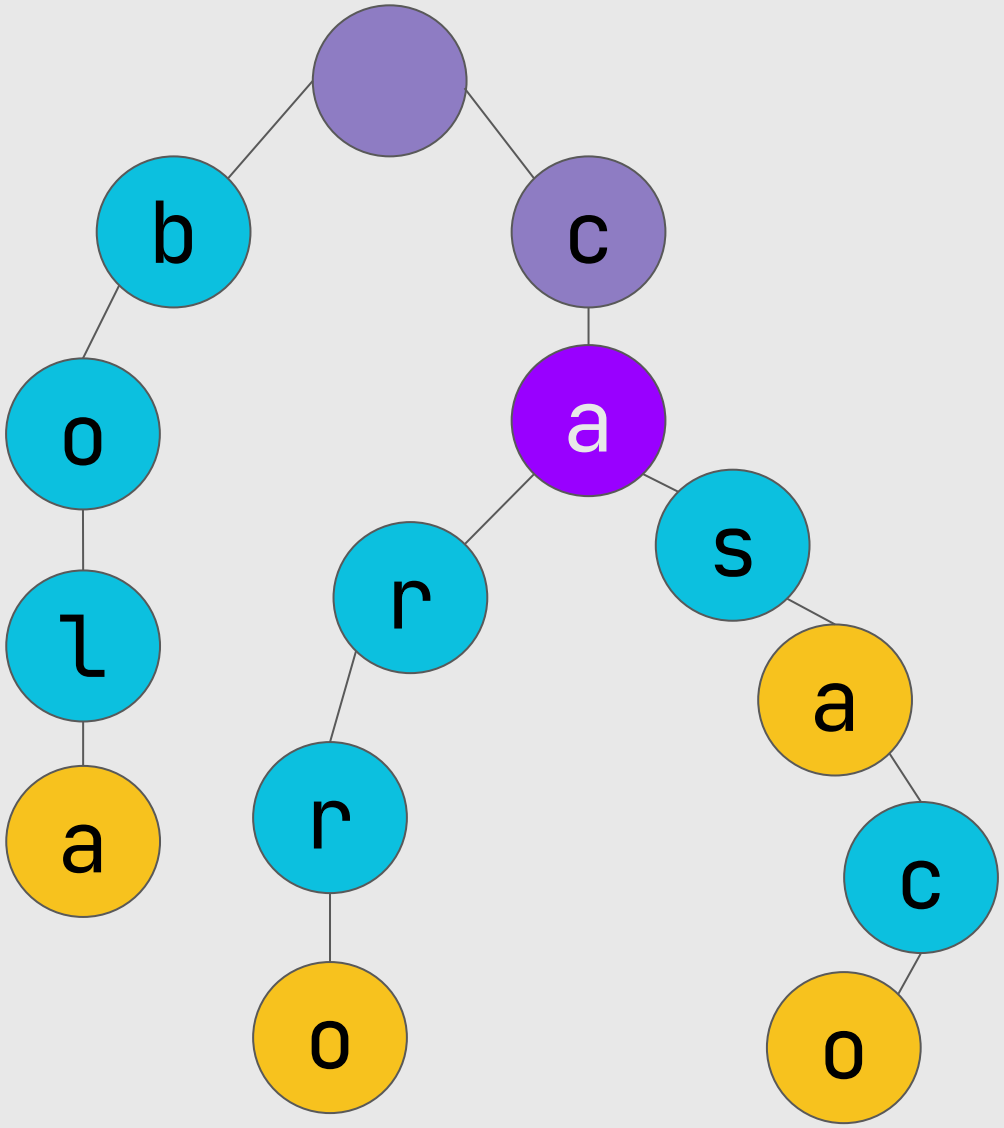
TRIE

Busca da string “carro”:



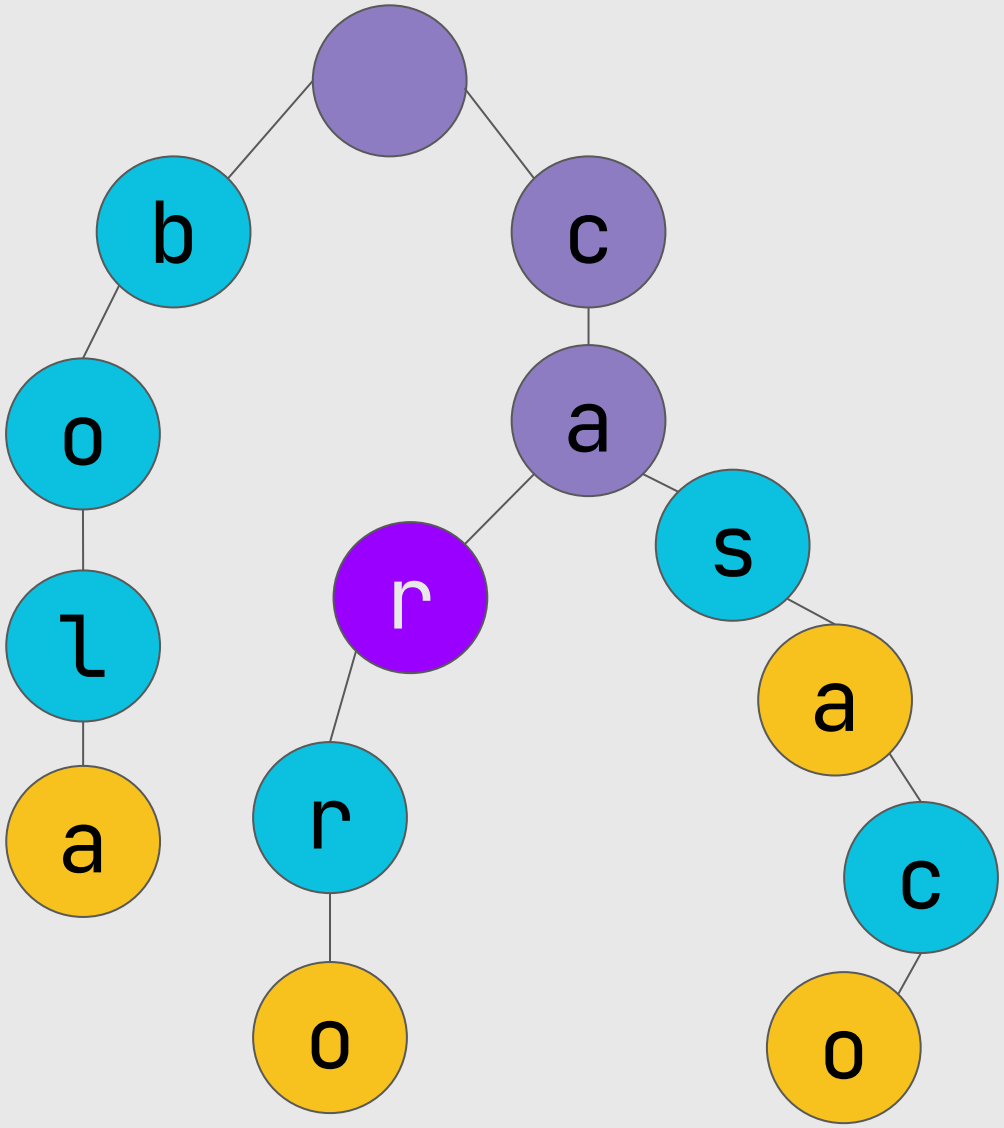
TRIE

Busca da string “carro”:



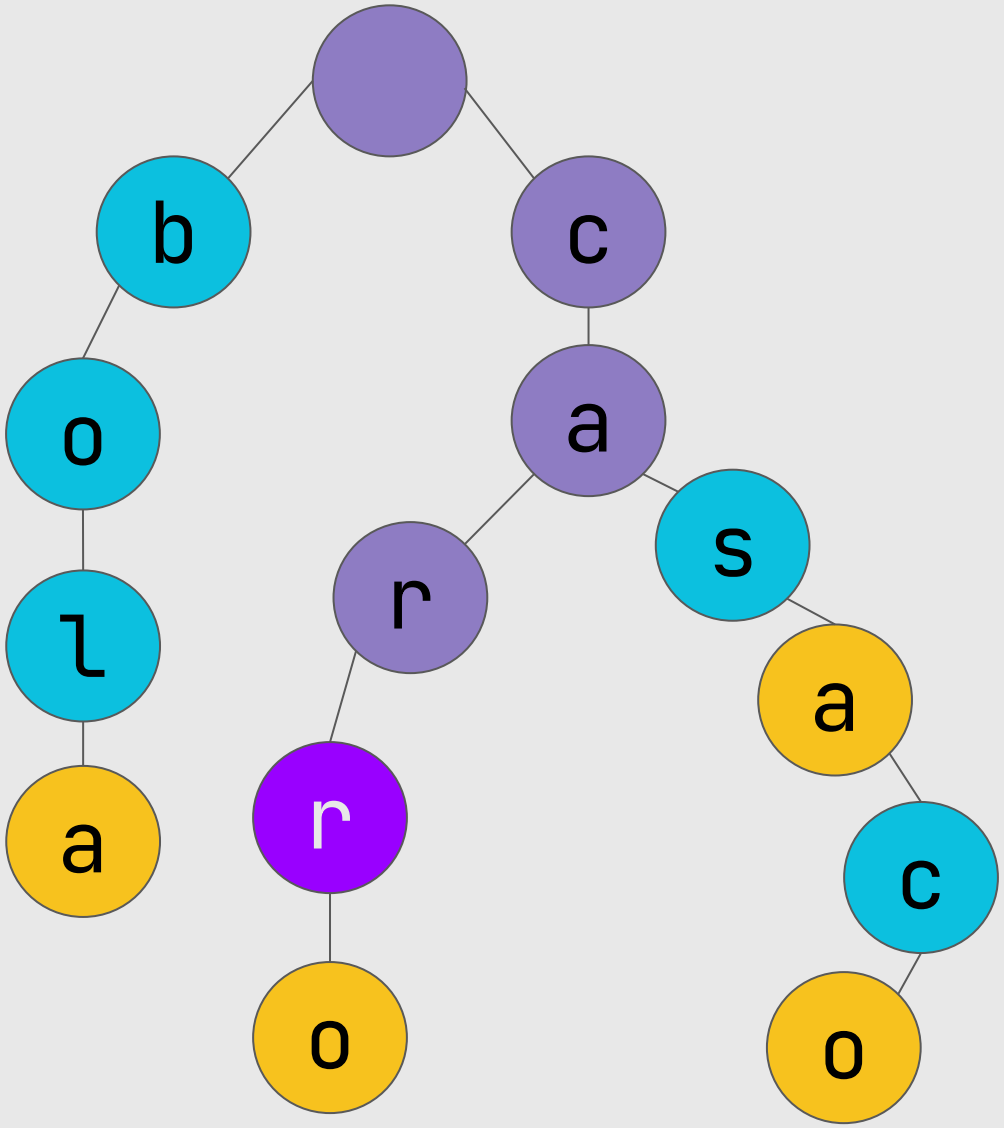
TRIE

Busca da string “carro”:



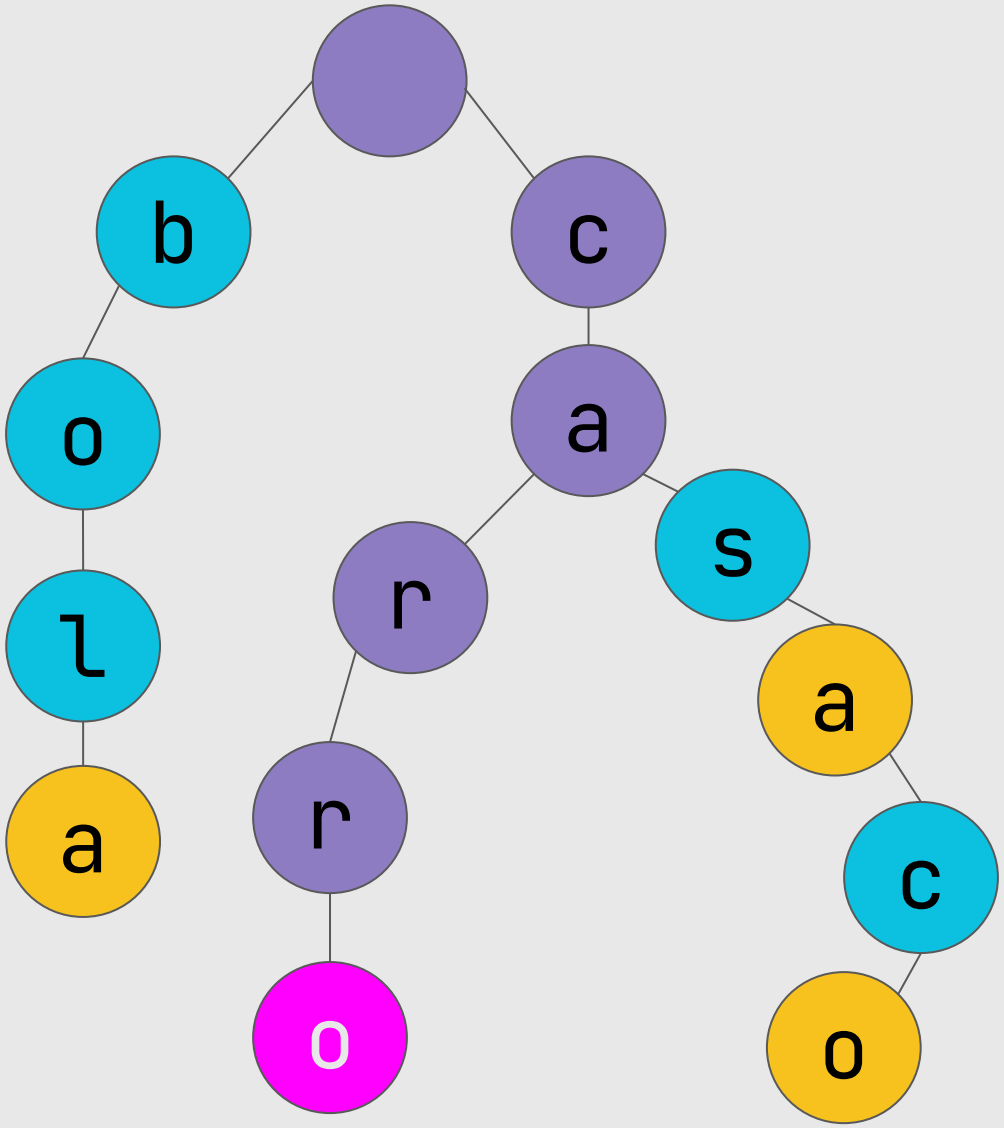
TRIE

Busca da string “carro”:



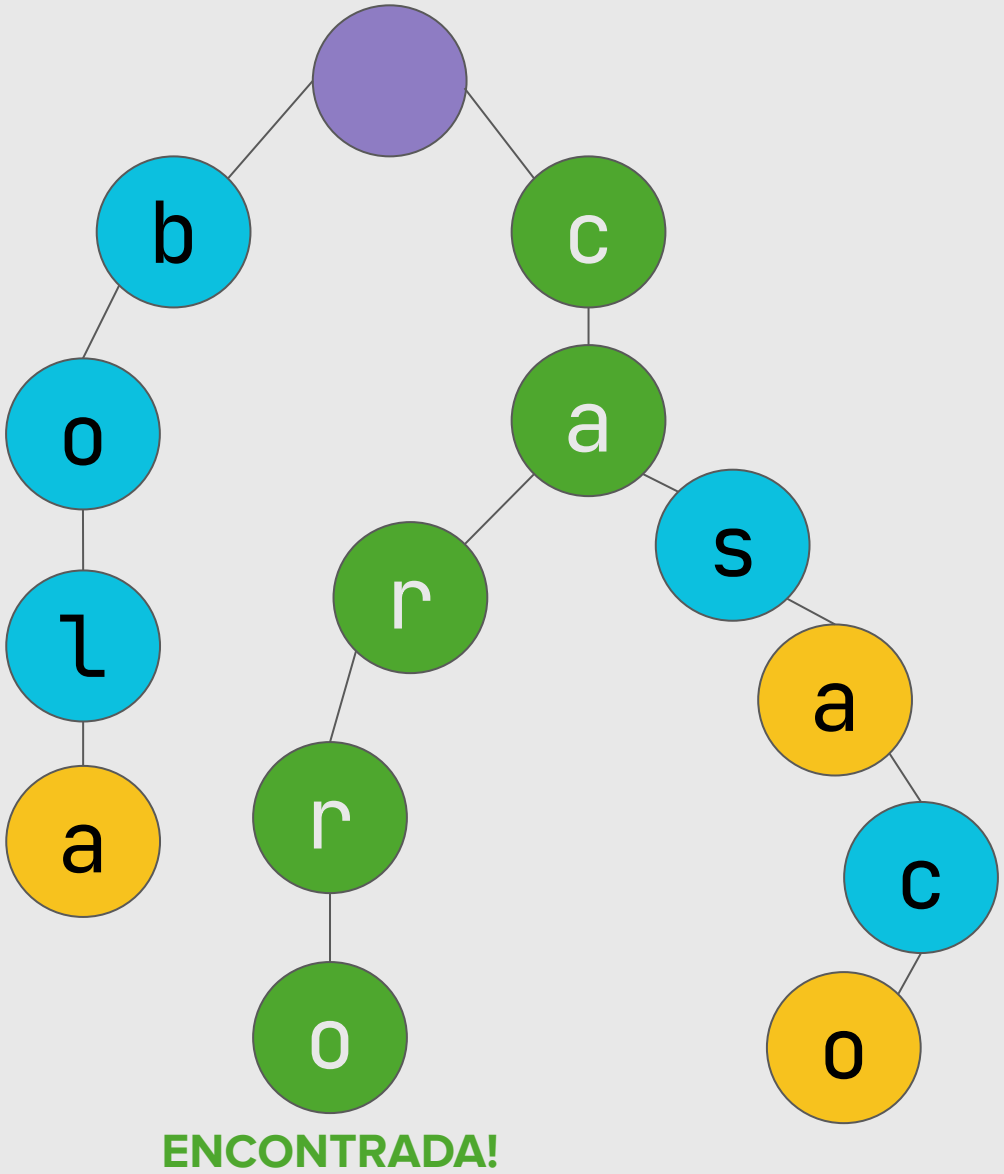
TRIE

Busca da string “carro”:



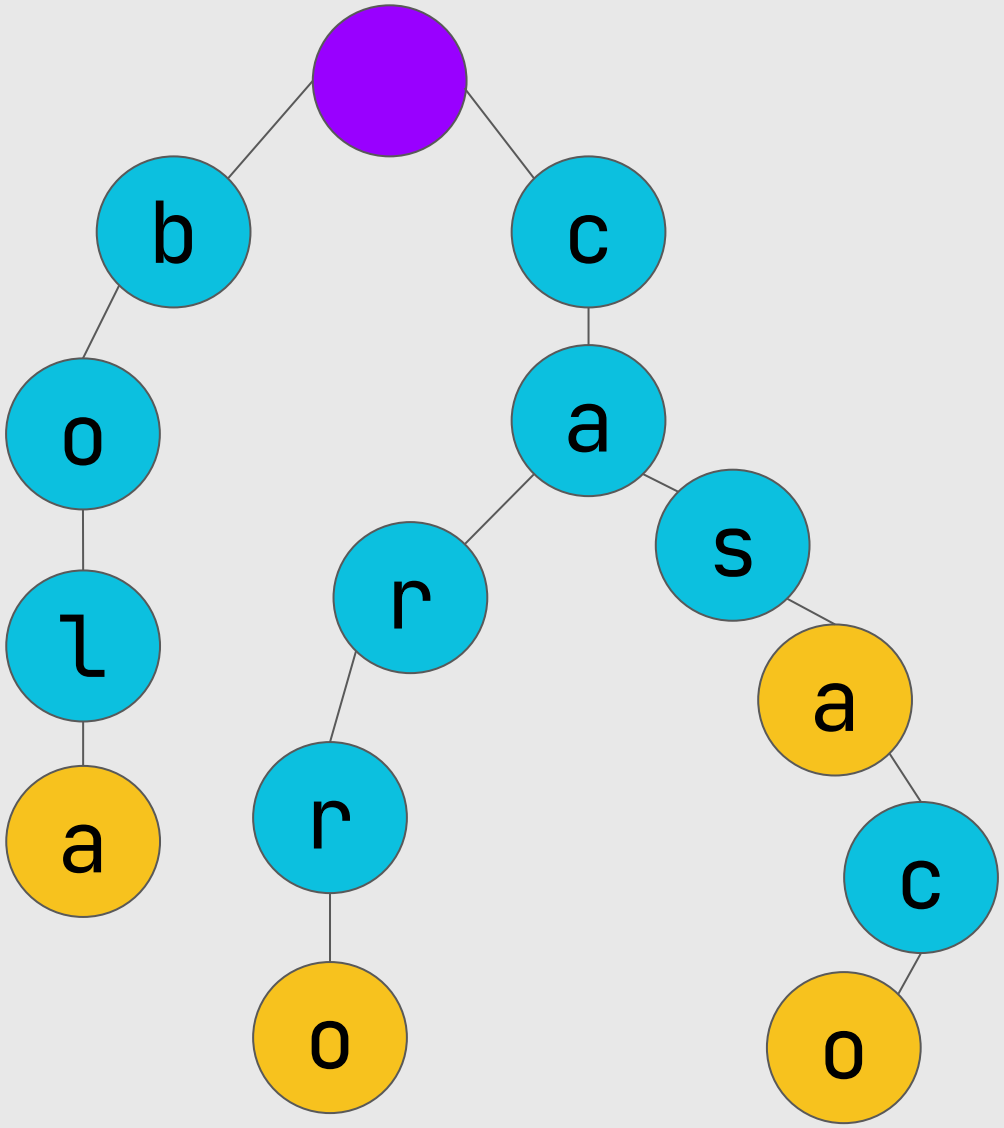
TRIE

Busca da string “carro”:



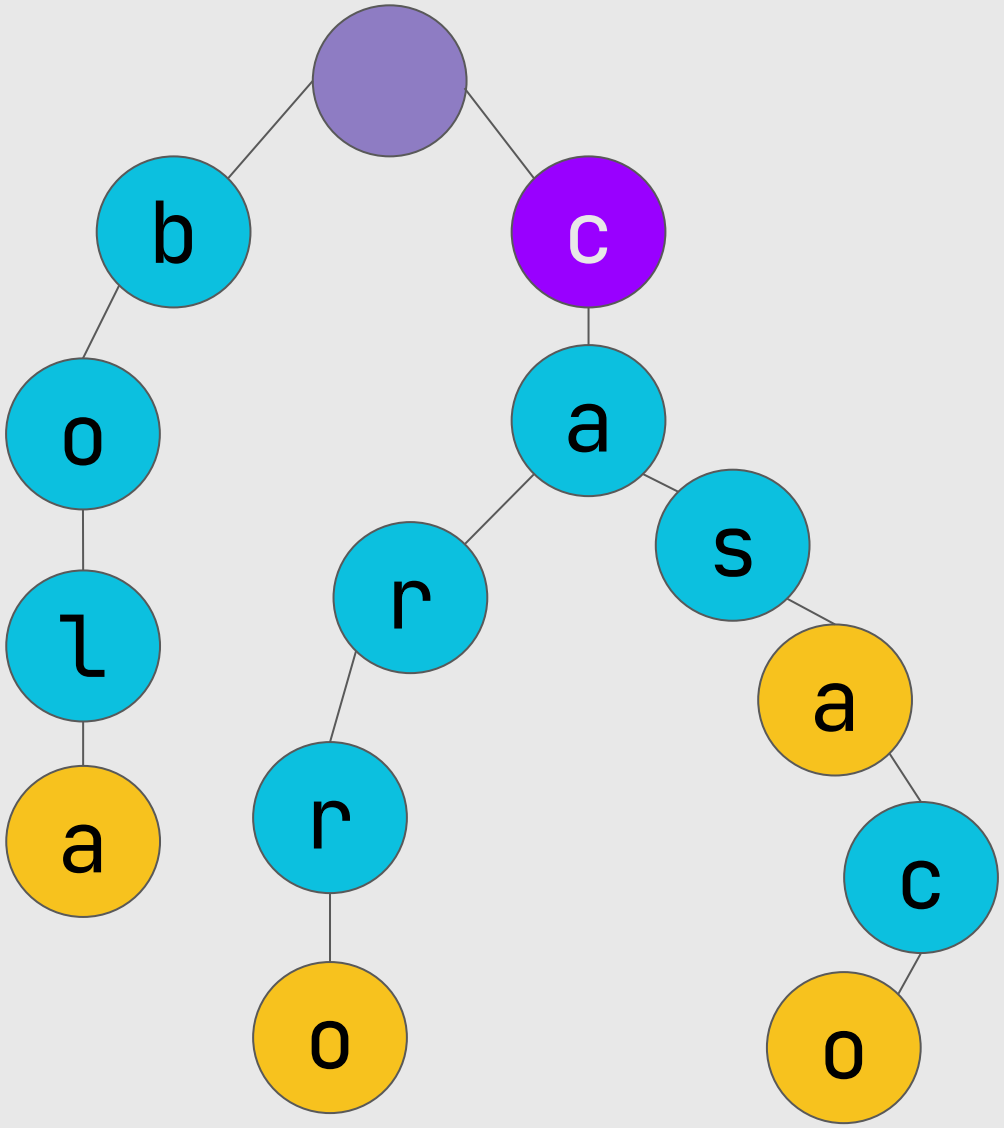
TRIE

Busca da string “carta”:



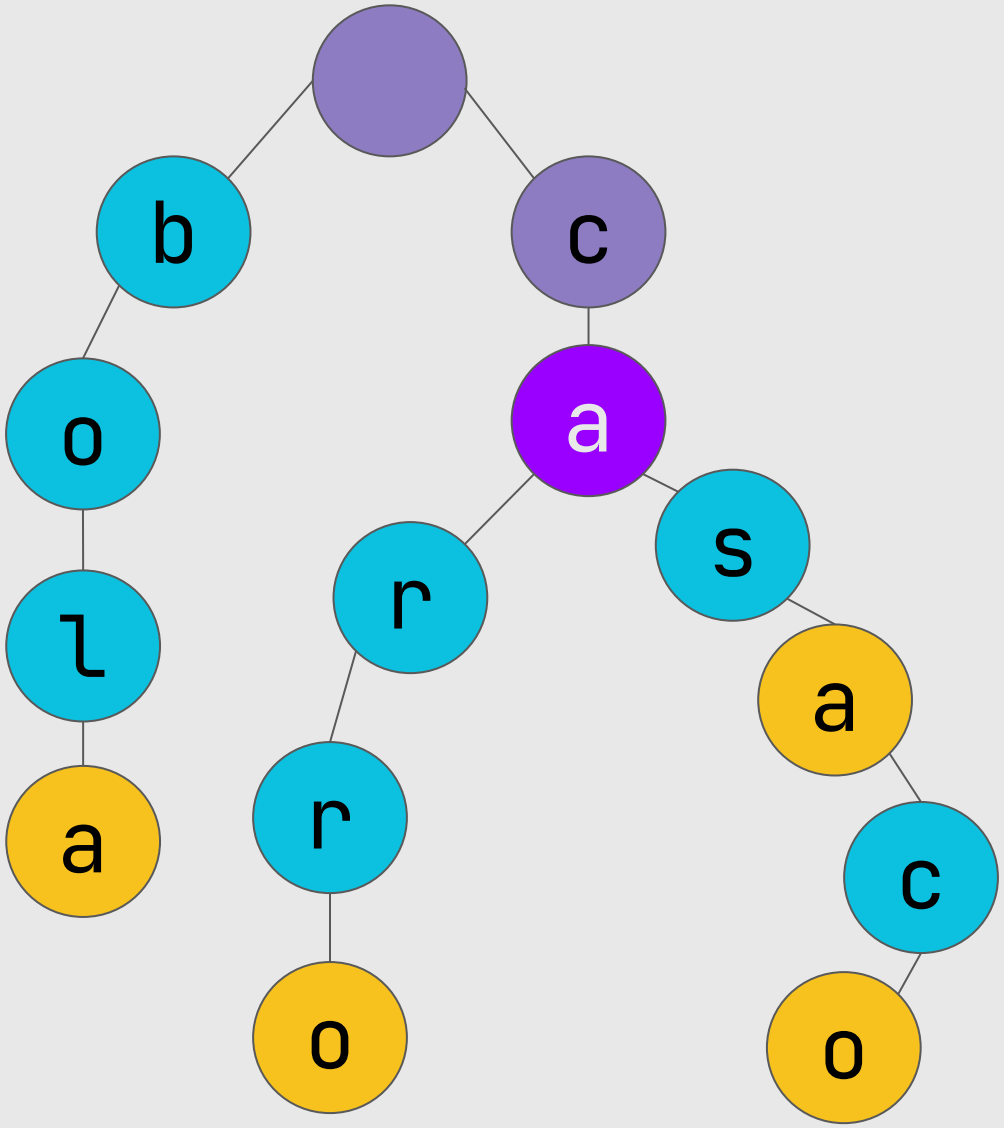
TRIE

Busca da string “carta”:



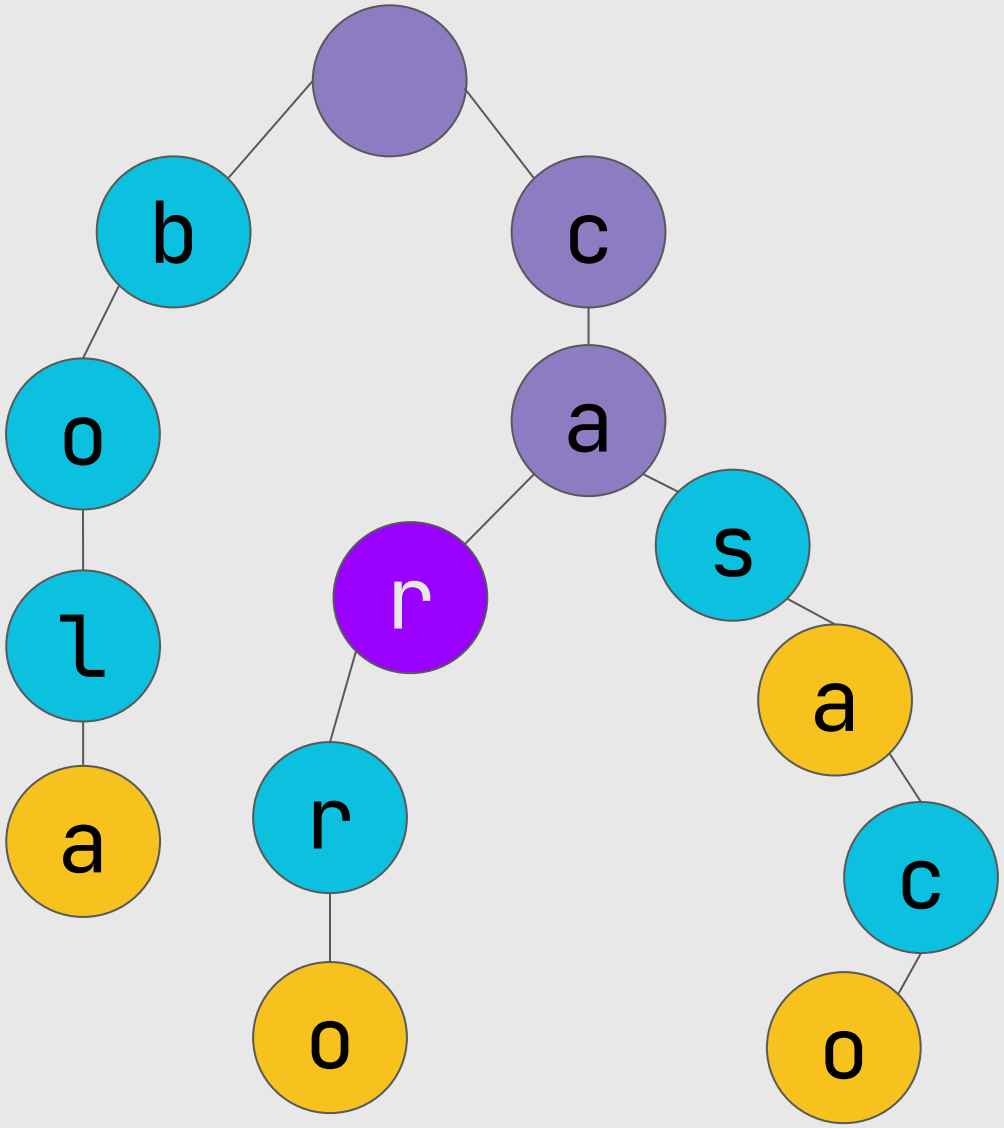
TRIE

Busca da string “carta”:



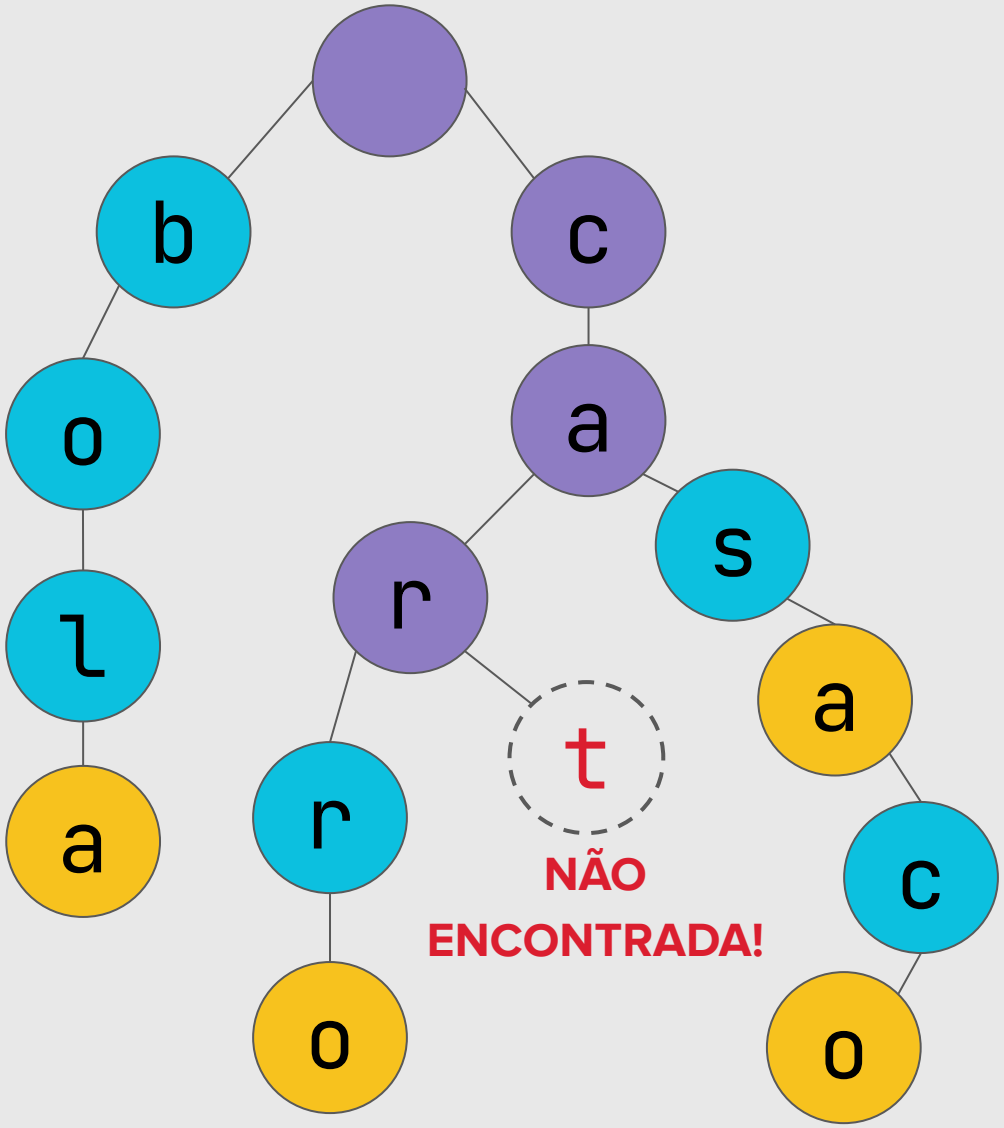
TRIE

Busca da string “carta”:



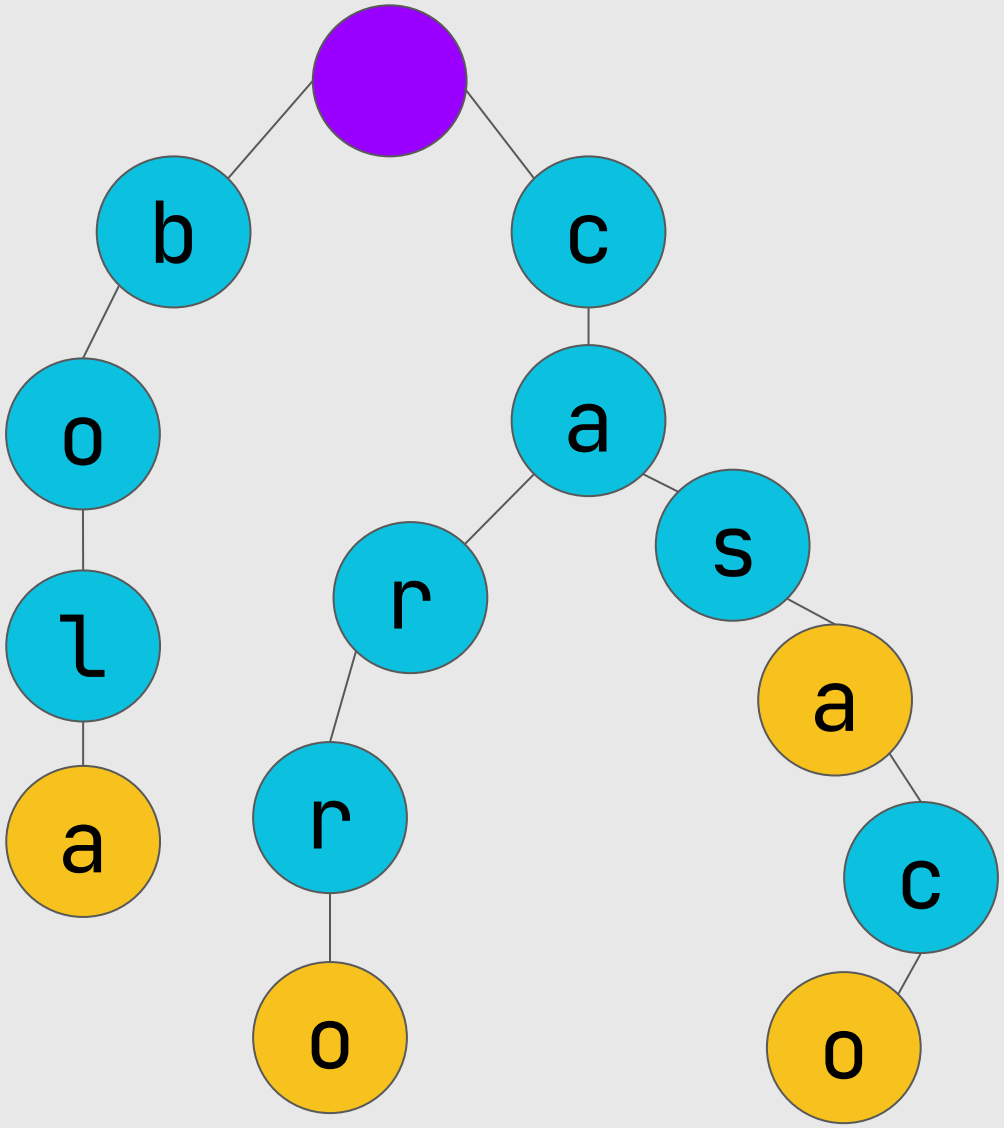
TRIE

Busca da string “carta”:



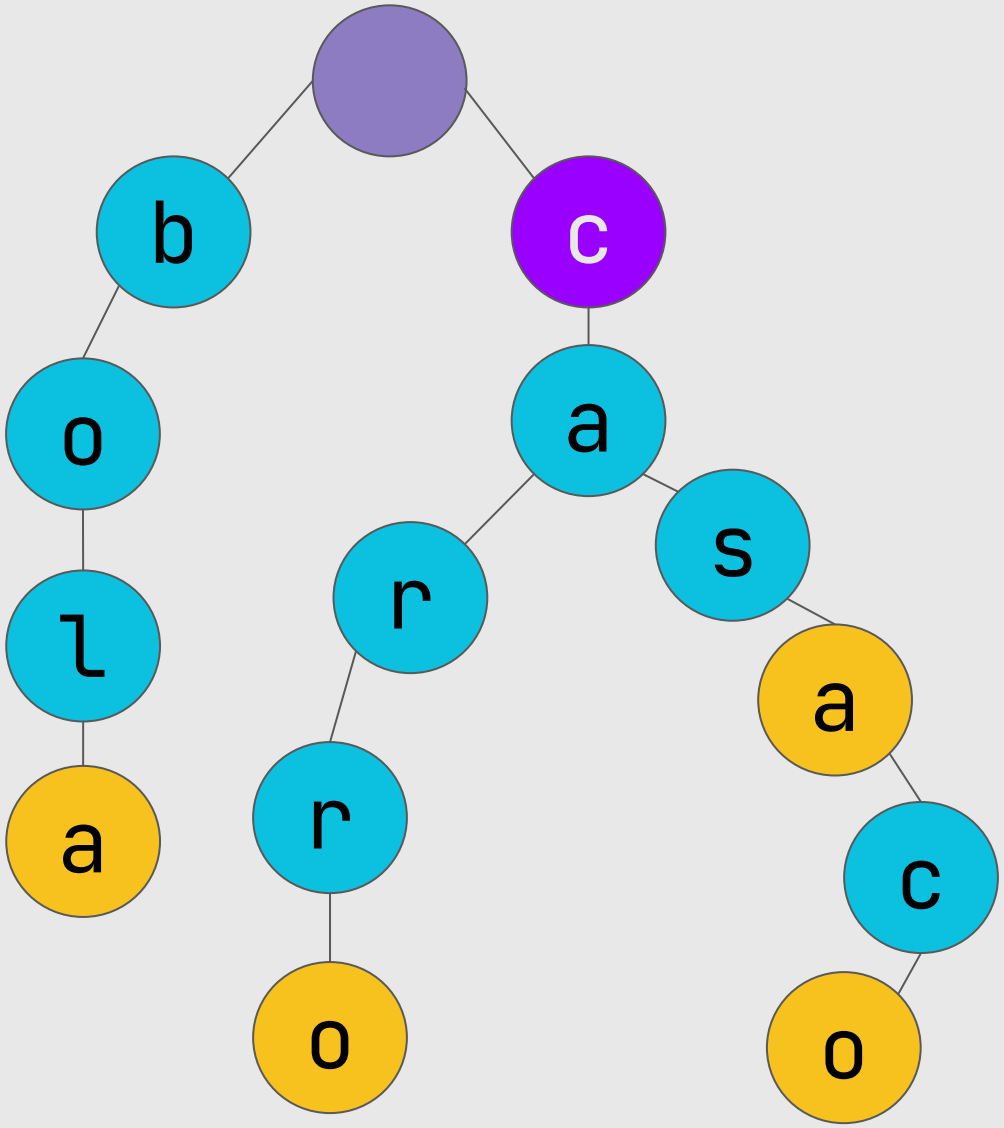
TRIE

Busca da string “car”:



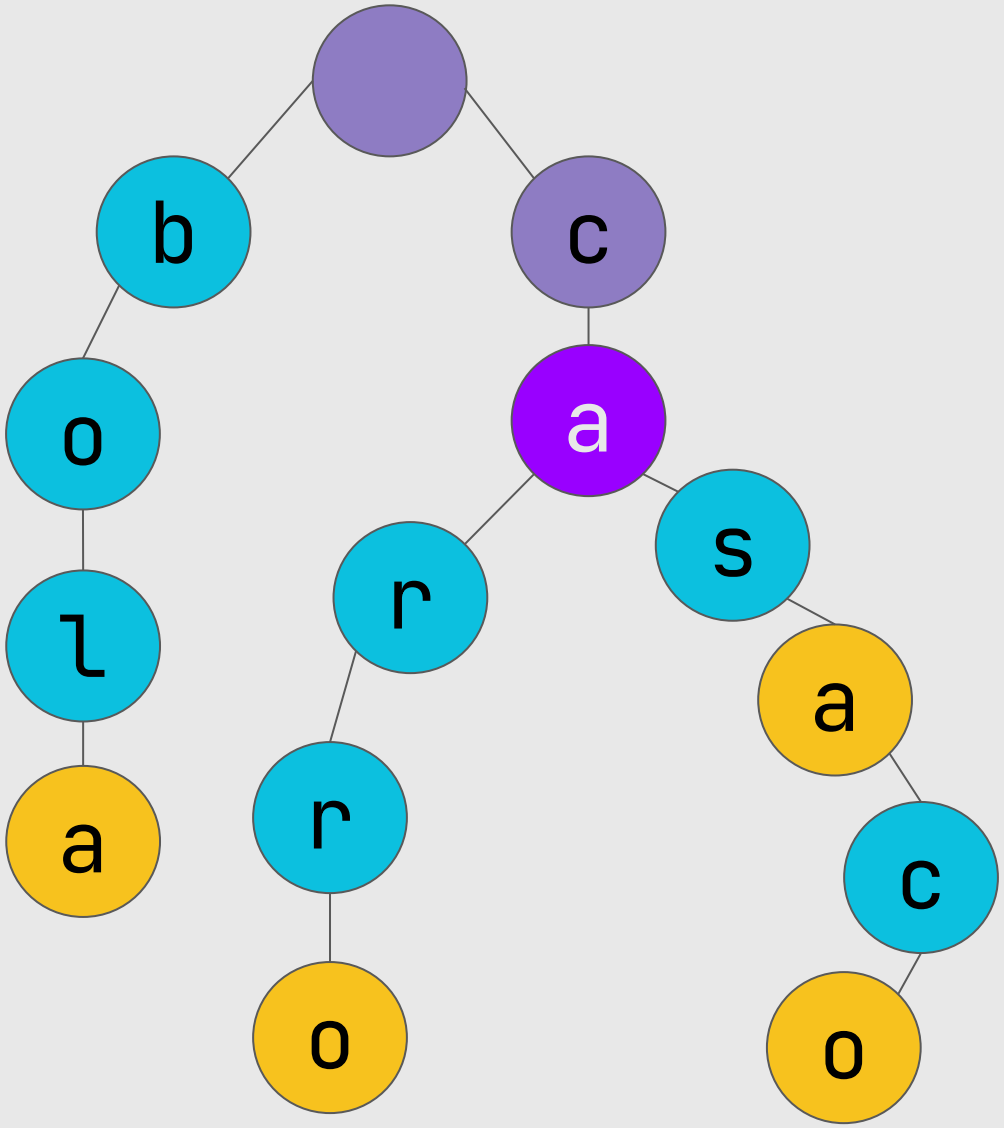
TRIE

Busca da string “car”:



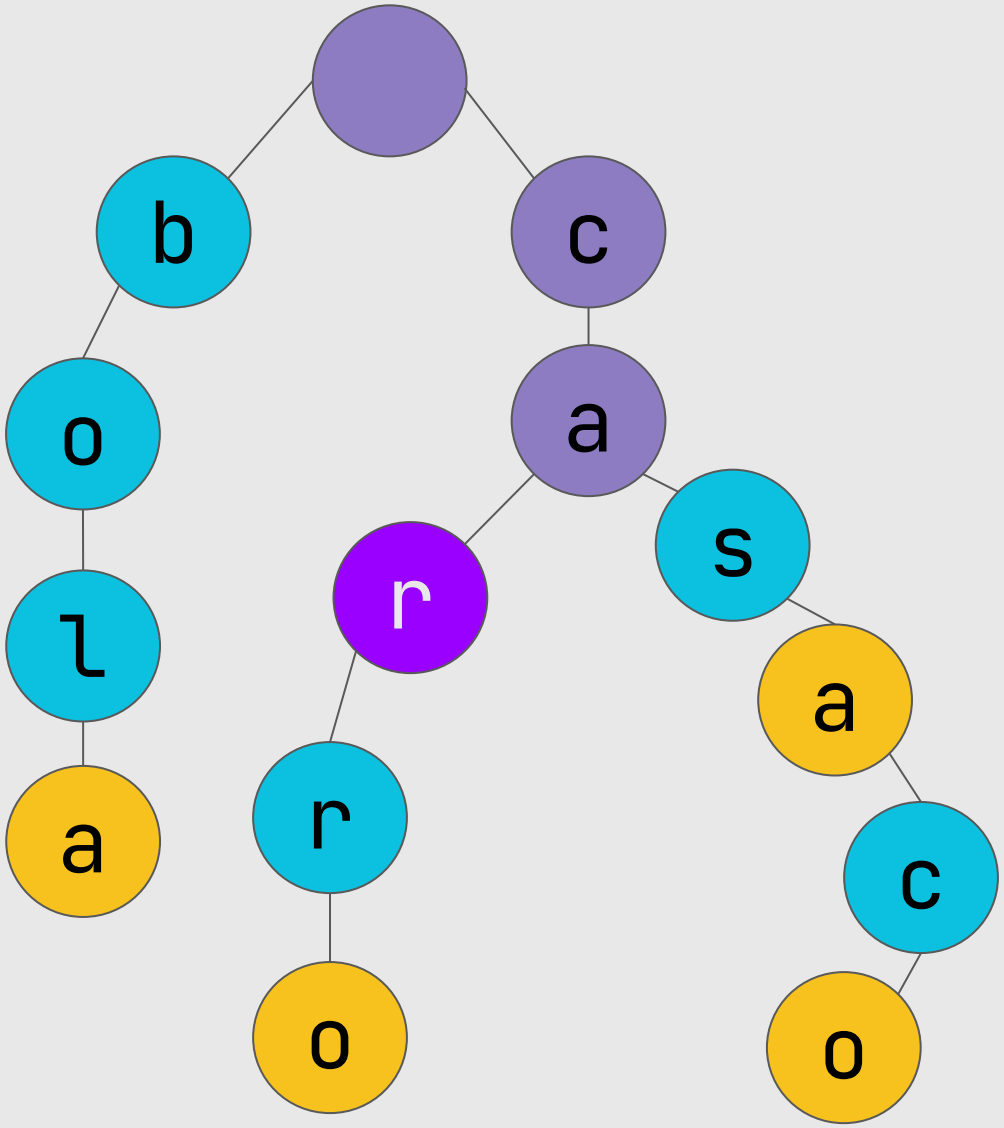
TRIE

Busca da string “car”:



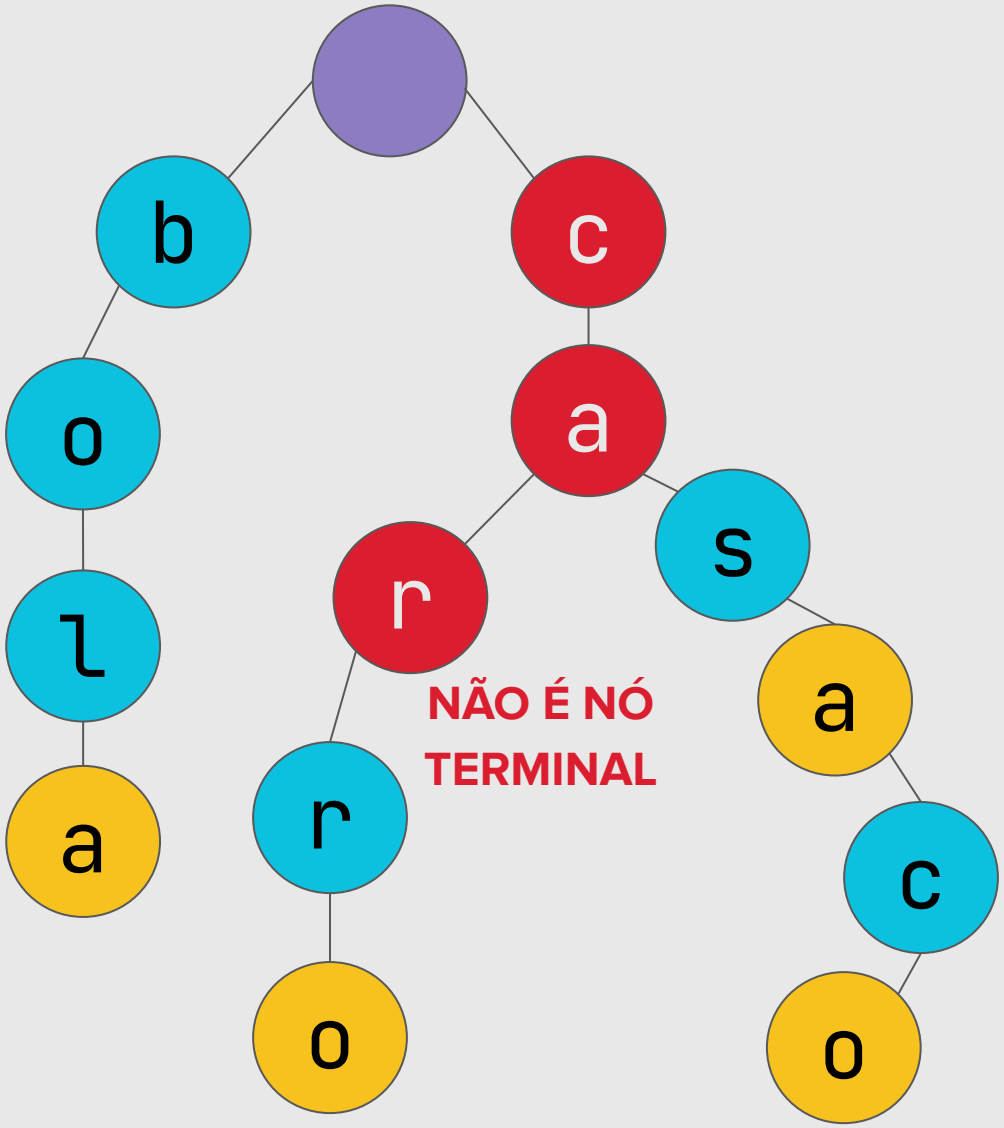
TRIE

Busca da string “car”:



TRIE

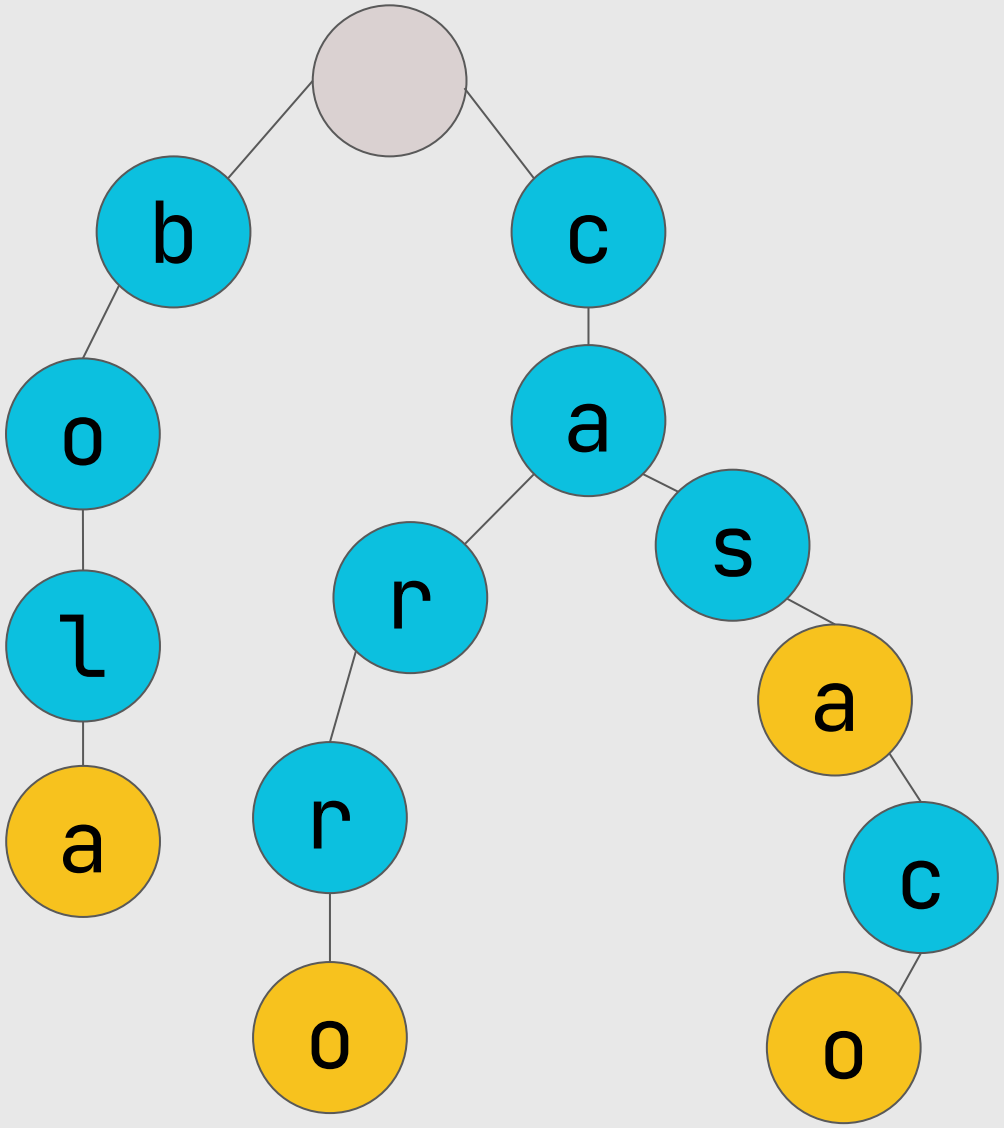
Busca da string “car”:



TRIE

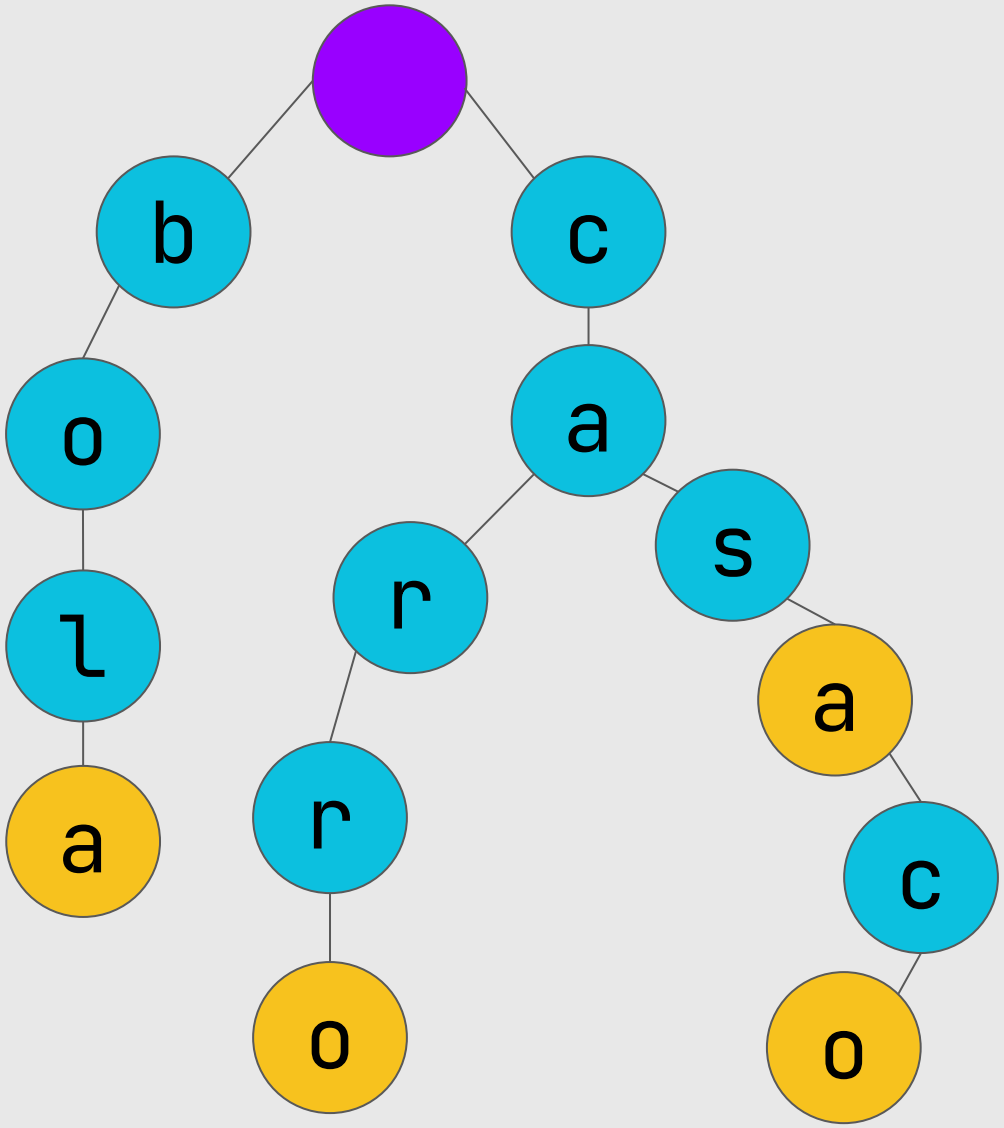
Para fazer inserções em uma TRIE, também precisamos apenas partir da raiz e navegar pelos nós desejados, mas dessa vez criamos novos nós sempre que necessário.

Caso os nós da TRIE estejam O(1), a inserção será em O(n). Caso os nós da TRIE estejam O(logn), então a inserção será em O(n*logn).



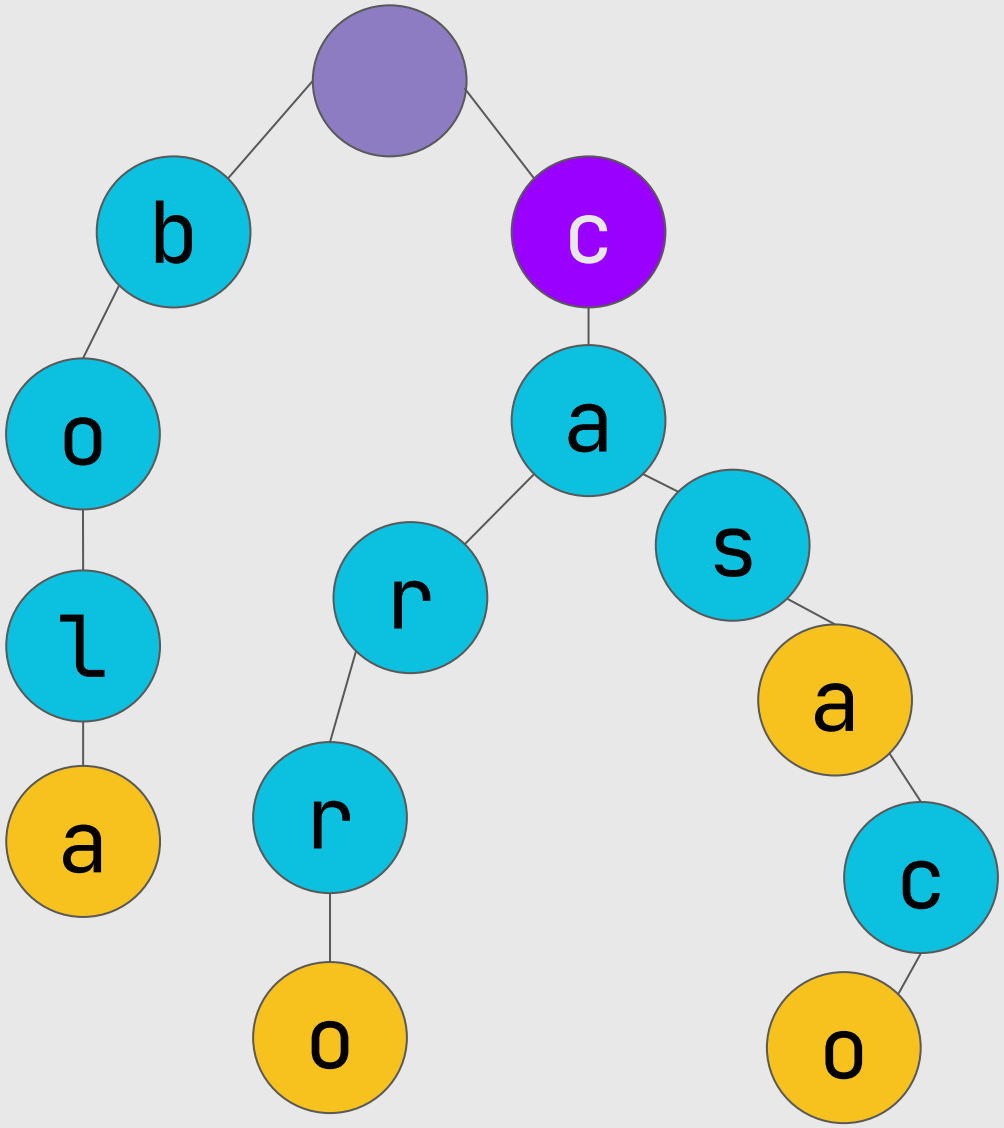
TRIE

Inserção da string “carta”:



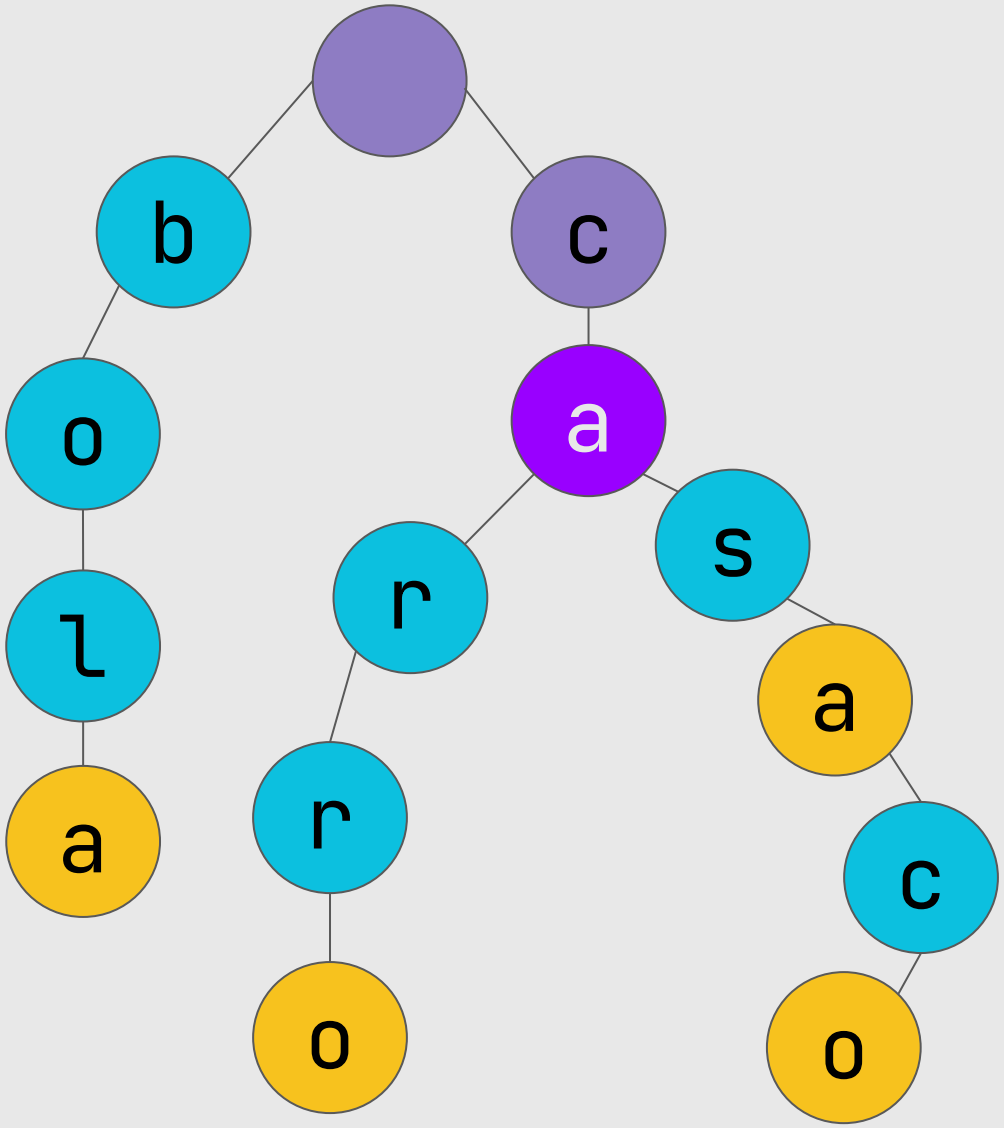
TRIE

Inserção da string “carta”:



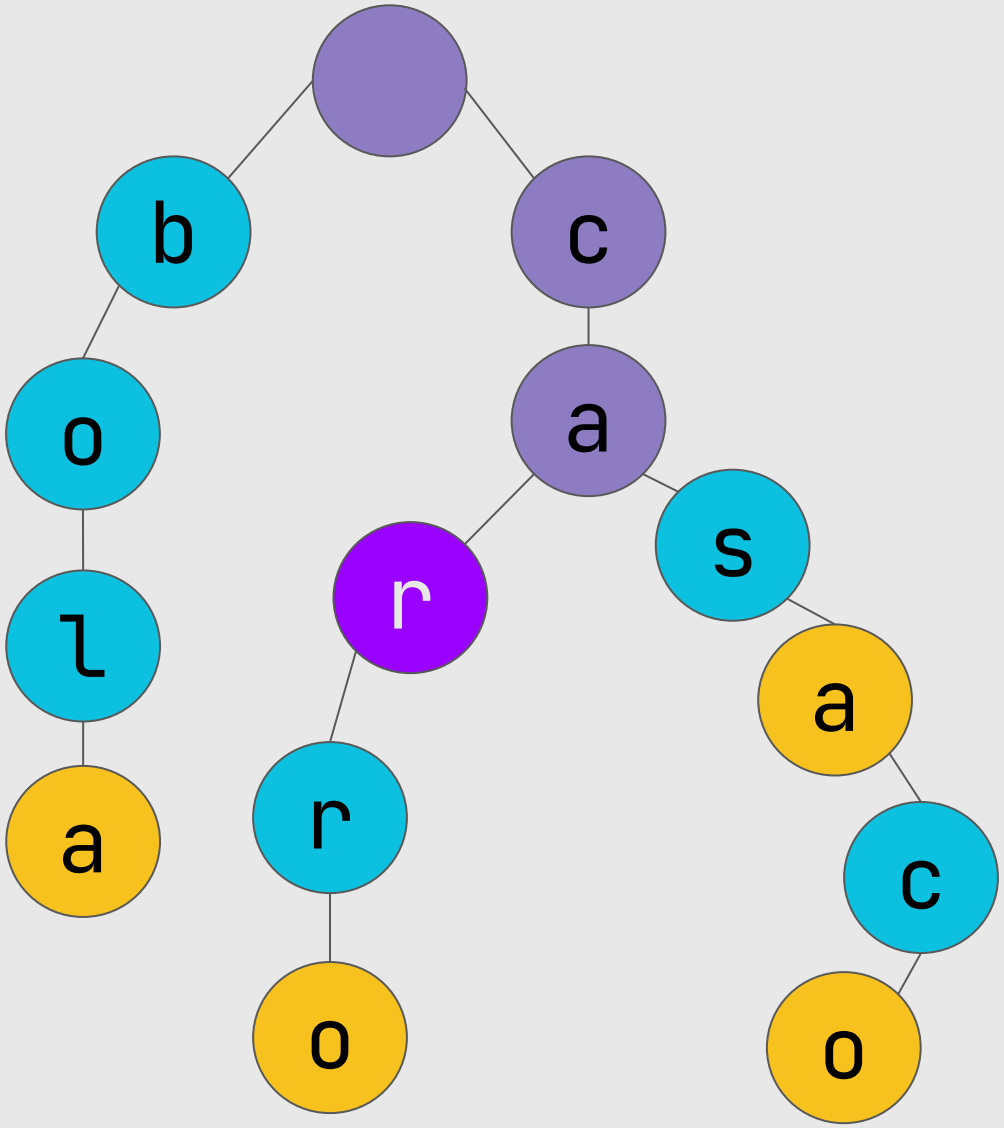
TRIE

Inserção da string “carta”:



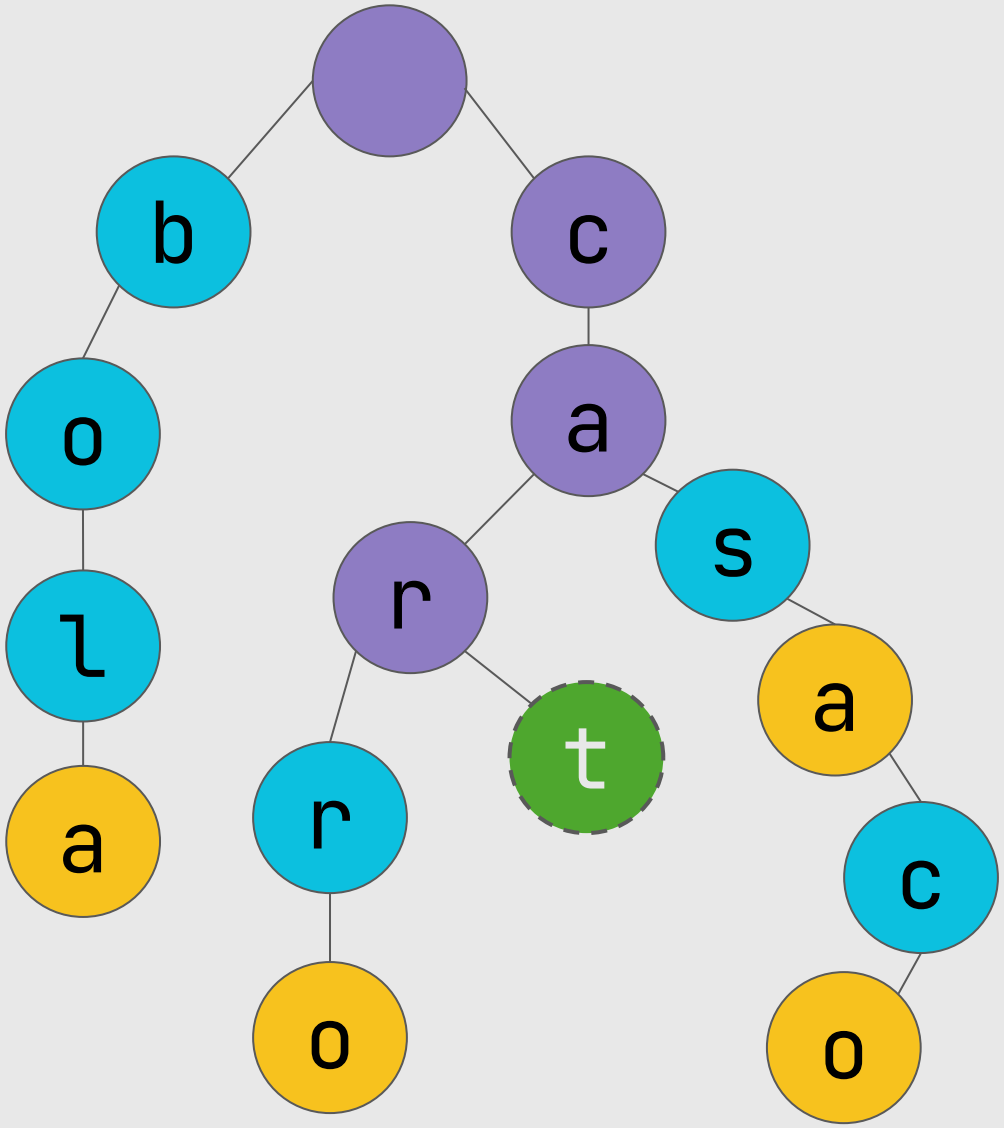
TRIE

Inserção da string “carta”:



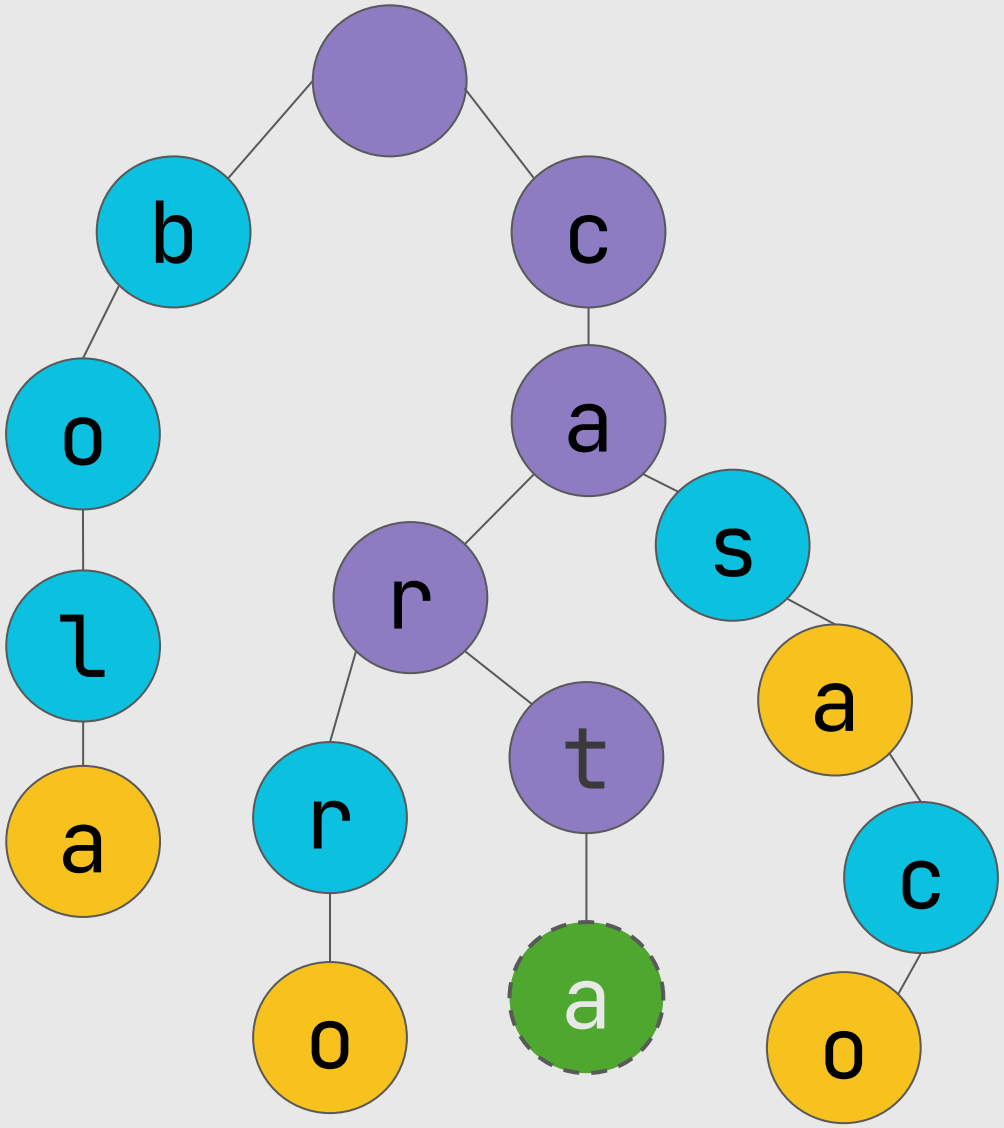
TRIE

Inserção da string “carta”:



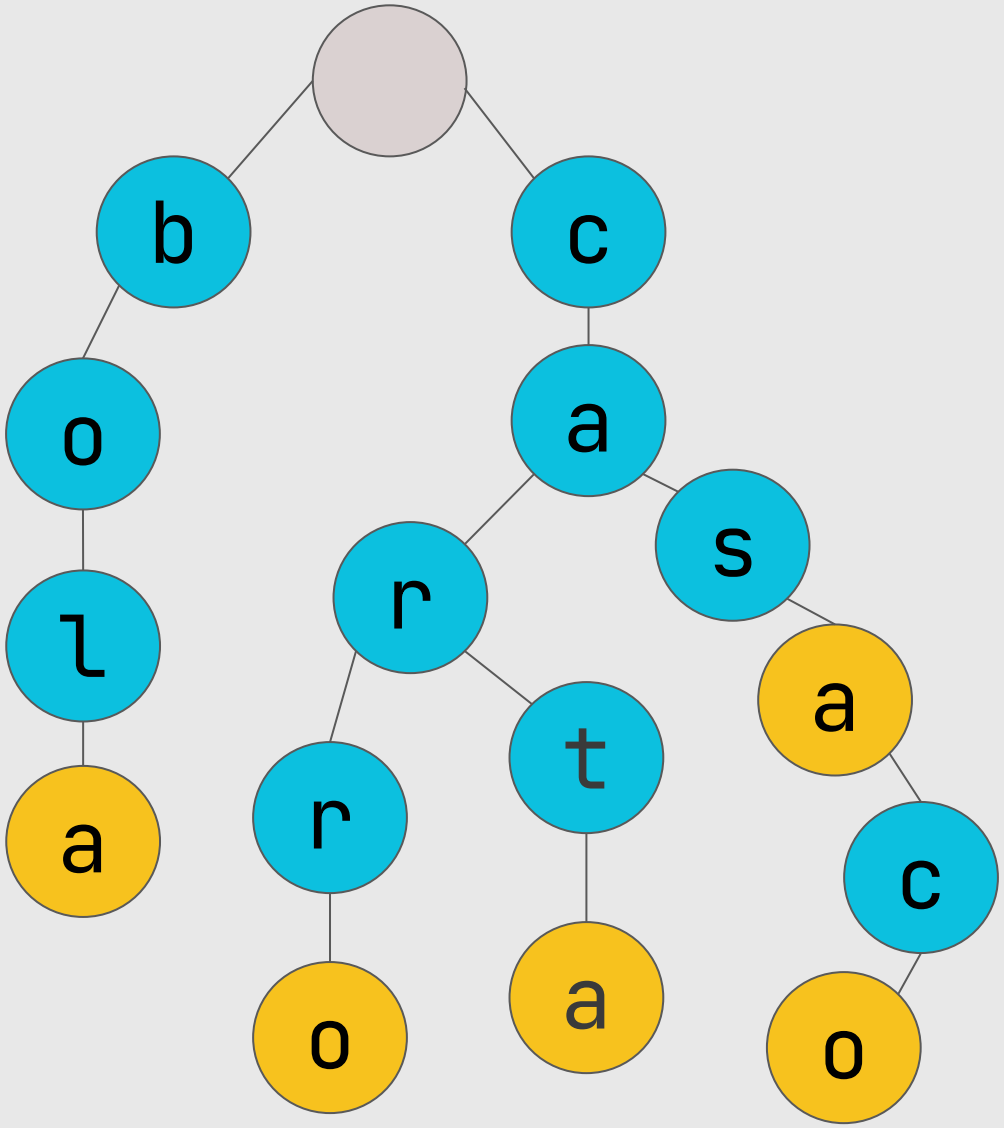
TRIE

Inserção da string “carta”:



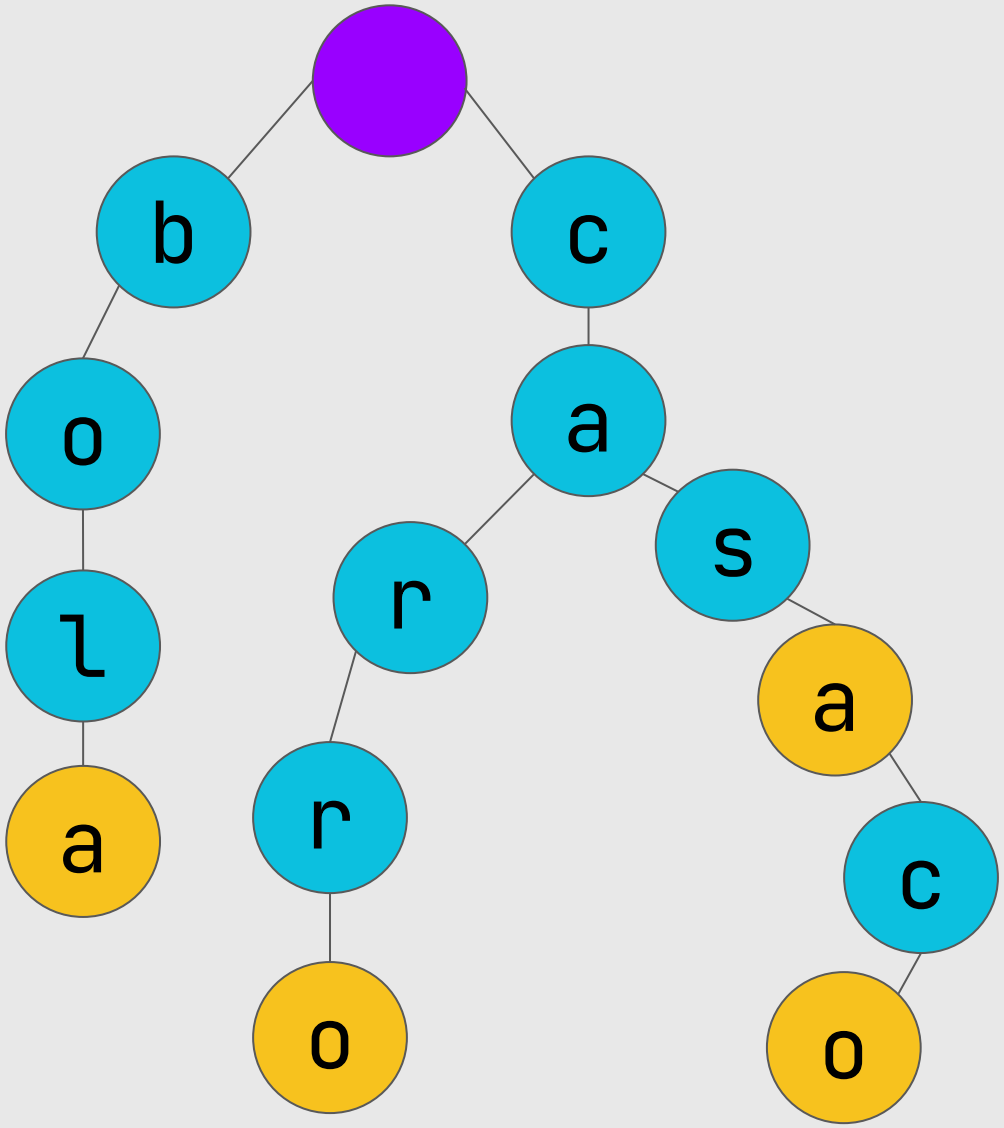
TRIE

Inserção da string “carta”:



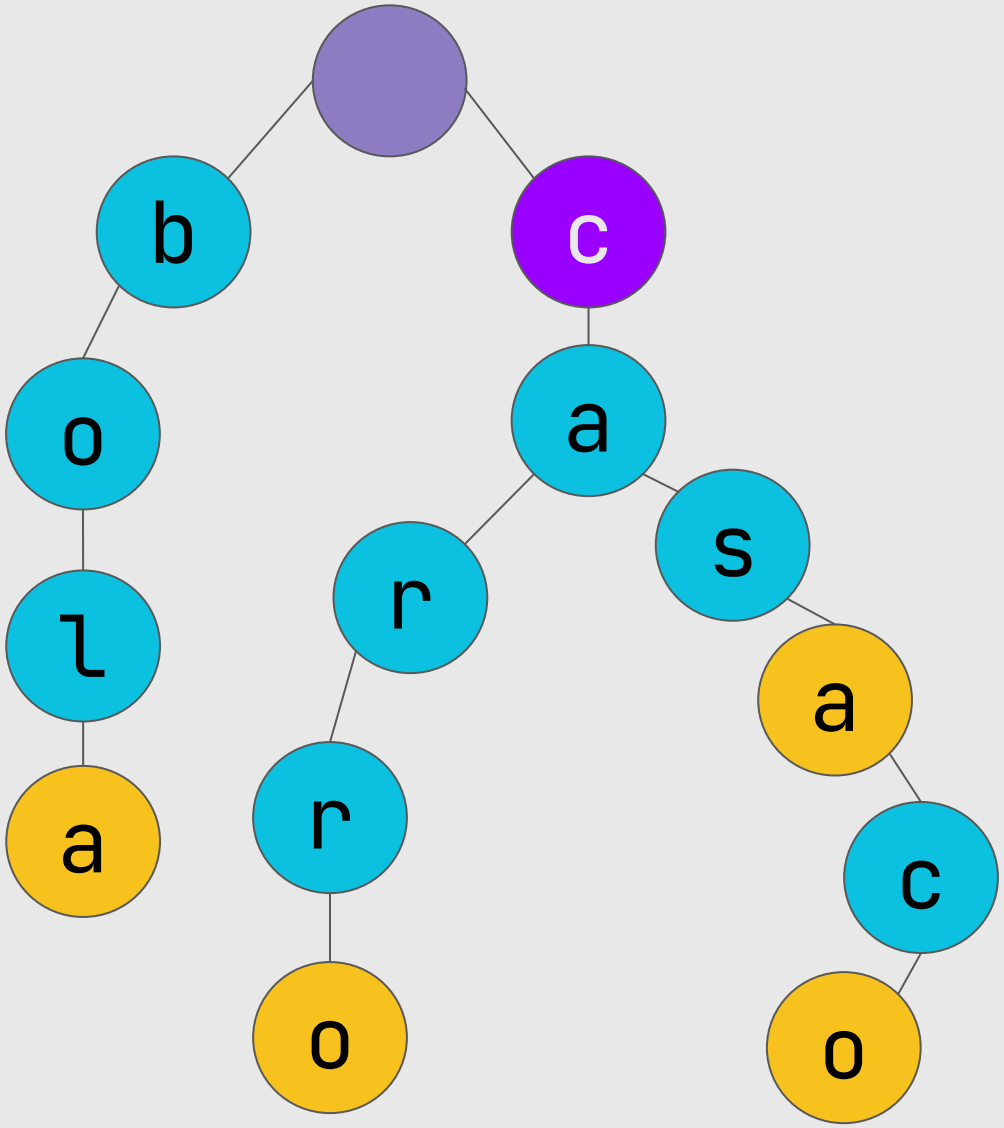
TRIE

Inserção da string “car”:



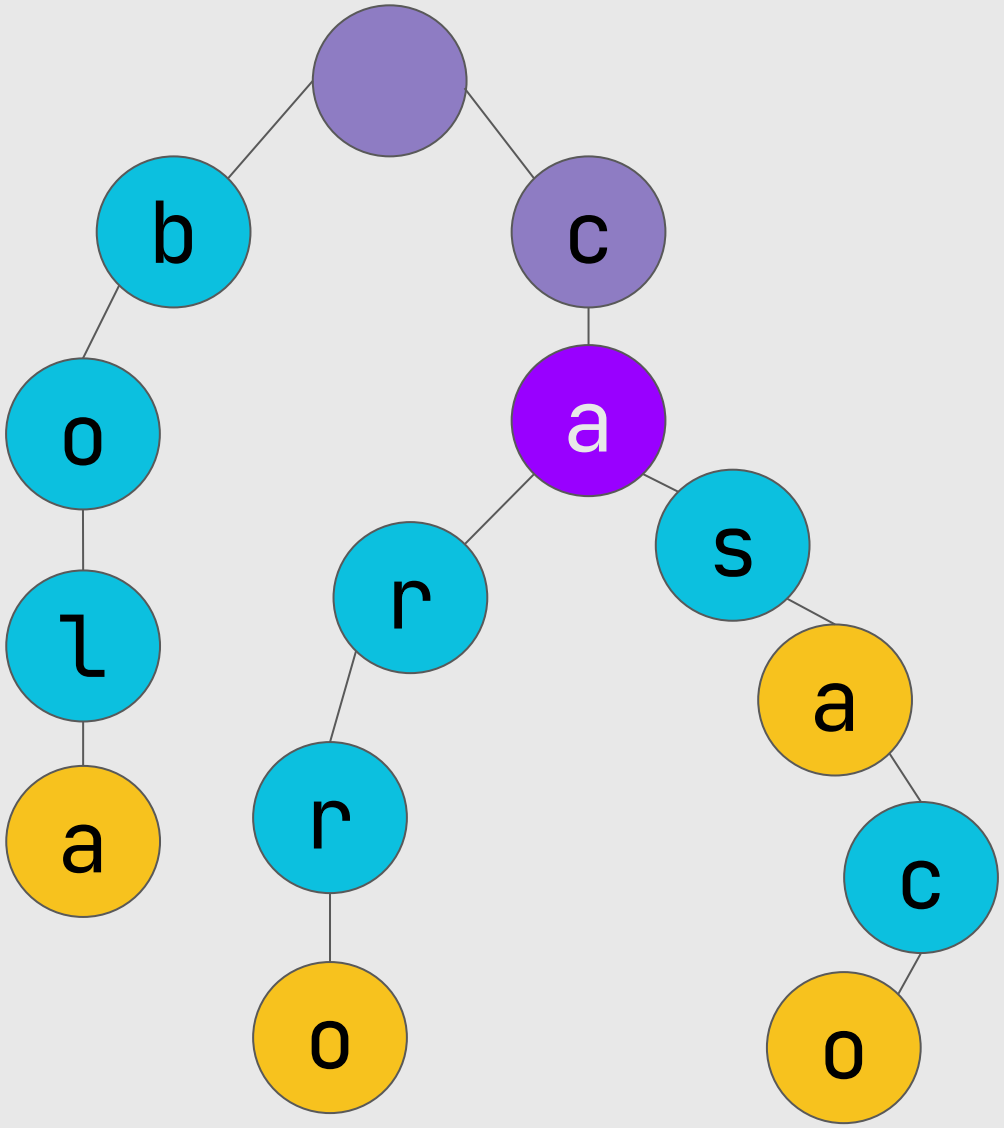
TRIE

Inserção da string “car”:



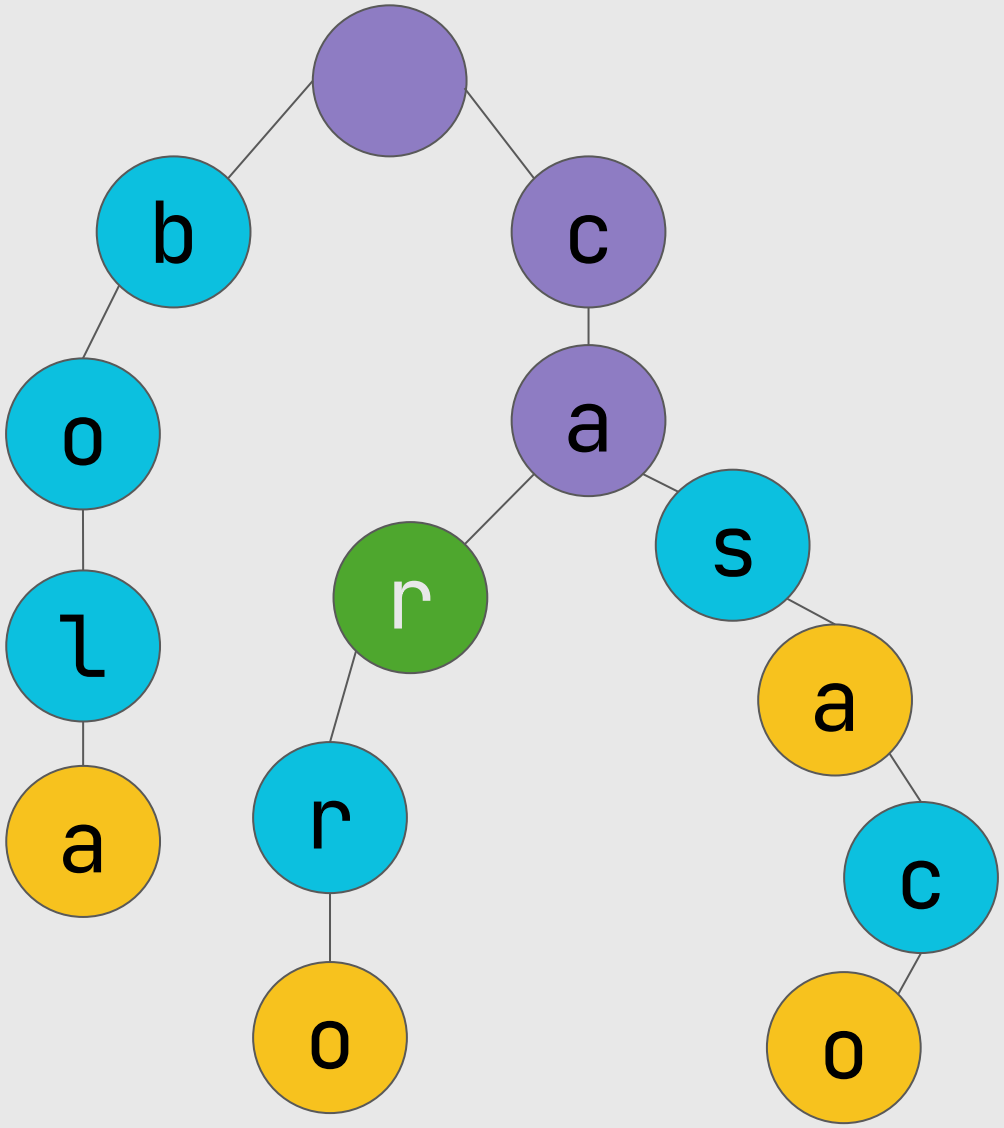
TRIE

Inserção da string “car”:



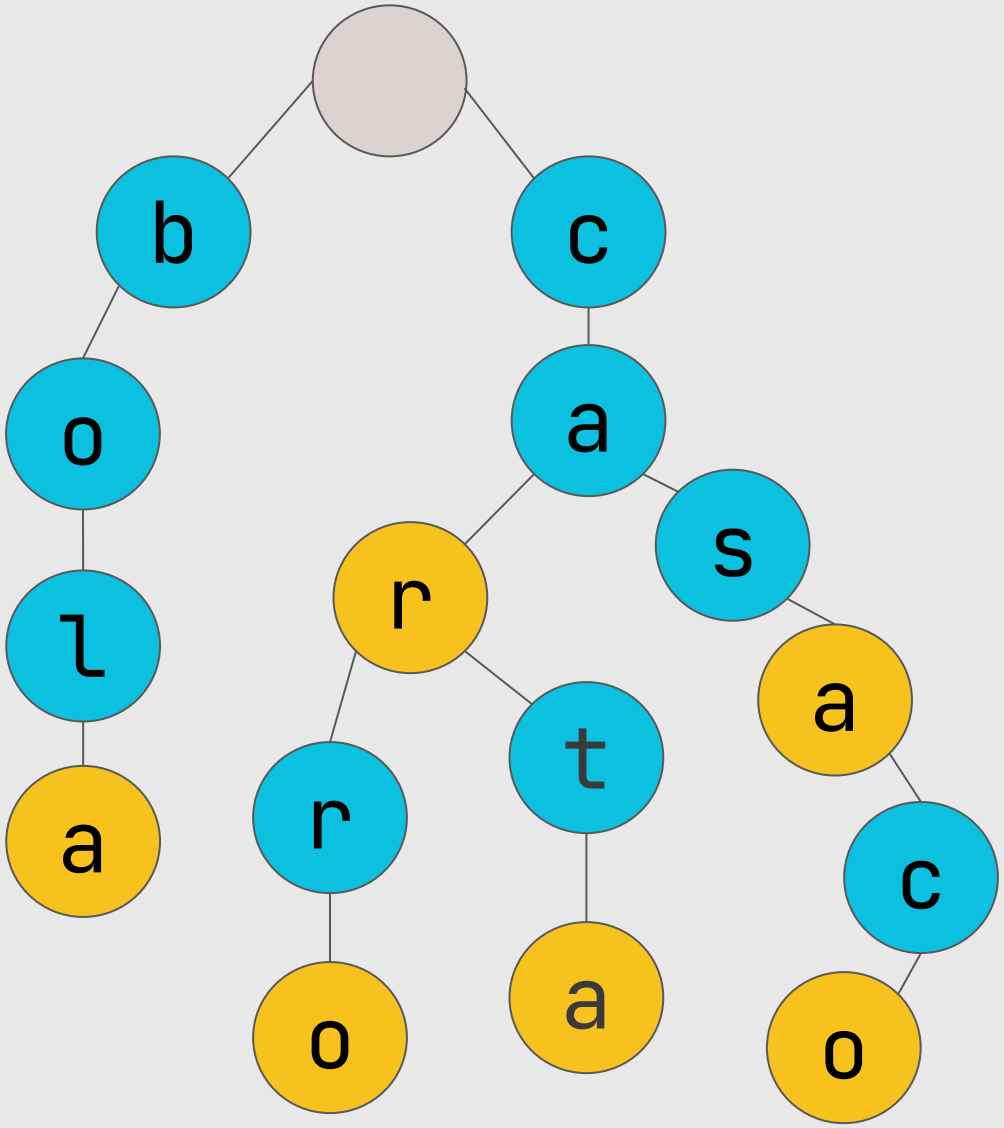
TRIE

Inserção da string “car”:



TRIE

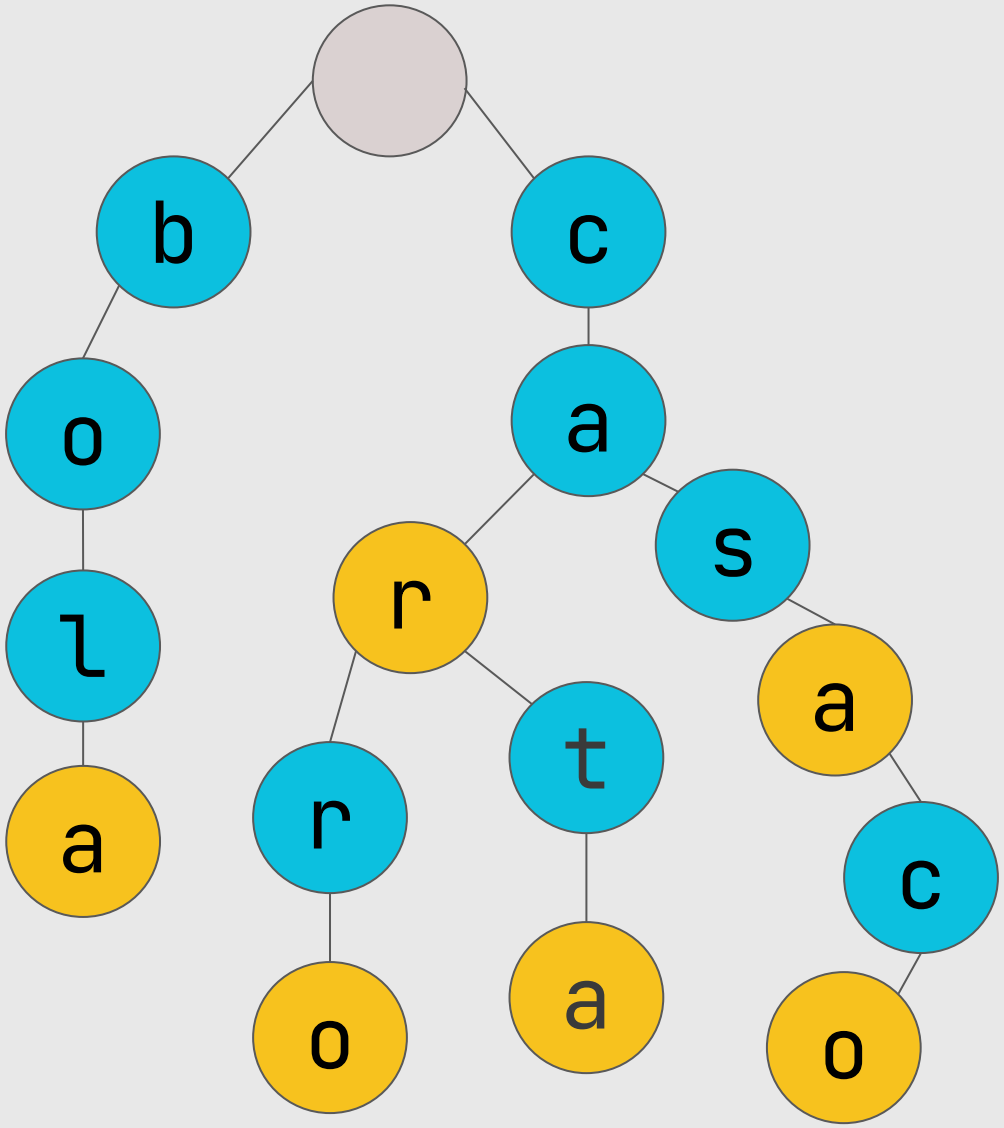
Inserção da string “carta”:



TRIE

Para fazer inserções em uma TRIE, também precisamos apenas partir da raiz e navegar pelos nós desejados, mas dessa vez criamos novos nós sempre que necessário.

Caso os nós da TRIE estejam O(1), a inserção será em O(n). Caso os nós da TRIE estejam O(logn), então a inserção será em O(n*logn).



MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

LCP (Longest Common Prefix)

Problema

Temos um problema onde precisamos achar o **maior prefixo em comum entre Q pares de strings, de tamanho N.**

Como superar O(Q*N)?

abcdabcd
abcdefgh

Resposta

MaratonaCIn Centro de Informática UFPE Universidade Federal de Pernambuco

Busca binária!

- Se o prefixo [0....i] é igual nas duas, sabemos que [0...i-1] também é igual.
- porém, se [0...i] for diferente, com certeza podemos dizer que [0...i+1] é diferente.

```
string a,b;
int i = 0, j = n;
while(j > i){
  int mid = (i+j+1)/2;
  if(a.substr_hash(0,mid) == b.substr_hash(0,mid)){
    i = mid; // [0..mid-1] iguais, resposta é pelo menos mid.
  }
  else{
    j = mid - 1; //se são diferentes, não pode ser mid ou maior.
  }
}
```

O(Q* log N)

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

Pi Array

Problema

MaratonaCIn Centro de Informática UFPE Universidade Federal de Pernambuco

$$S = "abcdabcdabcd"$$

Qual a menor substring que, concatenada consigo mesma múltiplas vezes, forma S inteira?

Solução

- Podemos iterar pelos possíveis tamanhos que dividem n, e testar com hash:
- No pior caso, são $n + n/2 + n/3...$ operações. O número abaixo de 10^7 com mais divisores tem 240 divisores.

≃O(n)

Existe algum método exato?

Solução

- **Pi Array:**

- $\pi[i]$ é simplesmente o tamanho do maior prefixo próprio da substring $S[0...i]$ que bate perfeitamente com o sufixo dessa mesma substring.
- Aponte para um índice qualquer e mostre as duas pontas se sobrepondo.

Período* de [0...i] = **`(i+1) - π[i]`**

```
s:  HoHoHo
pi: 001234

s:  abcdabcdabcd
pi: 000012345678
```

Implementação

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

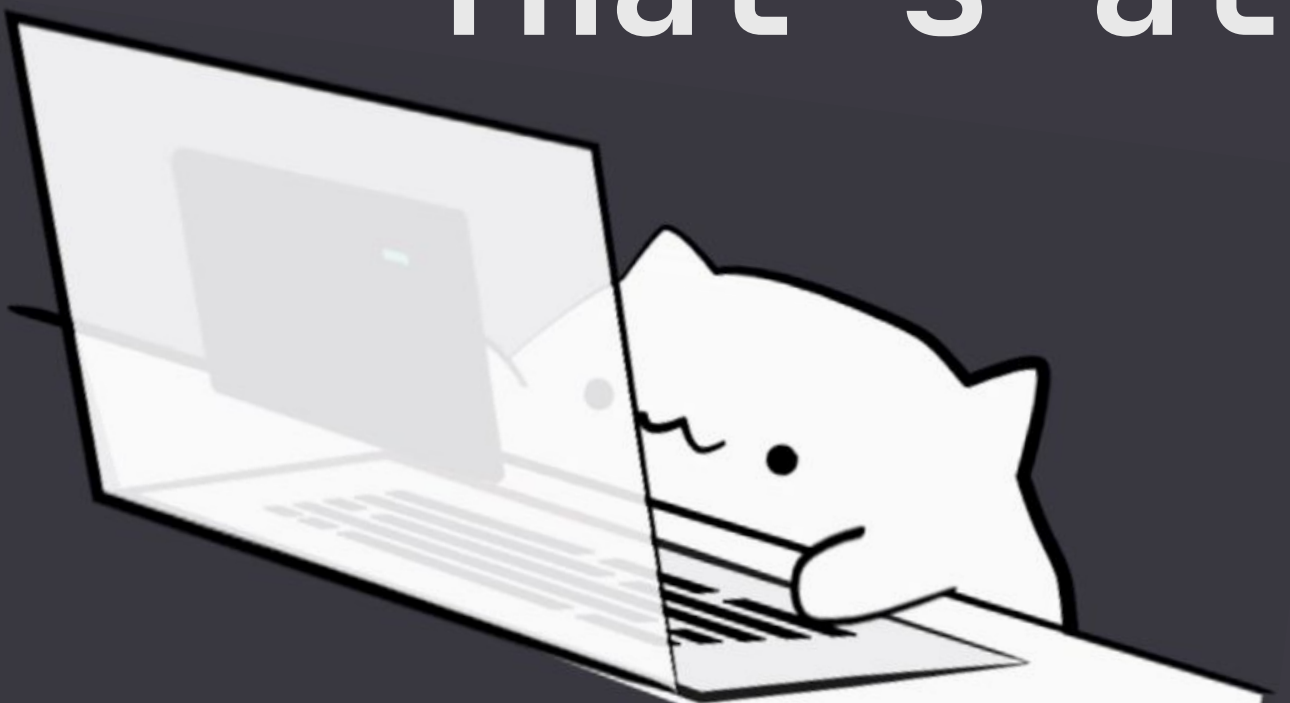
```
vector<int> Pi(string &t){
    vector<int> p(t.size(), 0);

    for(int i=1, j=0; i<t.size(); i++){
        while(j > 0  && t[j] != t[i]) j = p[j-1];
        if(t[j] == t[i]) j++;
        p[i] = j;
    }
    return p;
}
```

Dúvidas?

MaratonaCIn

Centro de Informática UFPE

Universidade Federal de Pernambuco

That’s all, Folks!